@python.hub

INTRODUCTION

→ In, JAVA a programmer should expres his logic thro class and objects only. It is not possible to write program without at least one class. This makes program lengthy.
for example, consider the program below :

```
/* JAVA program for printing Hello world */
class add{
    public static void main (string args[]) {
        system.out.println ("Hello World");
    }
}
```

→ But in certain cases not need to go for classes or objects and it is time consuming.

→ Rather they want to write c style coding as given below :

```
// C program to print Hello World
void main()
{
    printf("Hello World")
}
```

→ Here C program almost same as JAVA, no improvem in length of code.
Another problem for programmer, if he go for C, they will miss object orientation.

→ Programmer wants c style coding as well as JAVA style object orientation.

VISION

@python.hub

→ Python is a programming language that combines the features of C and JAVA.
It provide elegant style of developing program like C And also offers class and objects like JAVA.

→ For example, consider below python code:

```
# python program to print Hello world
print ("Hello World")
```

☺ see its that simple ☺

HISTORY OF PYTHON

→ Python laid its foundation in late 1980s.
→ Guido Van Rossum started implementation of python in December 1989, then later in February 1991 he published the code. (while JAVA developed in 1995)
→ He has developed python language to fill up gap between C and the shell.
→ Van Rossum picked the name python from TV show "Monty python's Flying circus"
→ Logo of python shows two intertwined snakes.
→ Python is open source means any body can freely download it and use it to develop programs.

VISION

@python.hub

PYTHON VERSIONS.

Python 1.0 - Introduced in Jan 1994
Python 2.0 - Introduced in oct 2000
Python 3.0 - Introduced in Dec 2008.
Python 3.7 - Came in June 27, 2018
Python 3.8 - Came in October 14, 2019

Note:
In python there is no backward support.
Python 3 never going to support to python 2.
Python 2 never going to run in python 3.
(some features may not support)

→ For example, To print "Hello World"

Syntax in python 2: print "Hello World"
Syntax in python 3: print ("Hello World")

COMPANIES USING PYTHON

QUORA
edX
UBER.
Instagram
Pinterest .....

VISION

@python.hub

FEATURES OF PYTHON

There are variety of reasons python gaining good popularity in programming community. Some of them are :—

1. Simple : It is simple programming language.
   → When we read python program we feel like reading English sentences. Hence developing and understanding program become very easy.

2. Easy to learn : Python uses very few keywords. It's program structure is simple and it resembles C language.
   → Most of the language constructs in C are also available in python.
   → Hence migration from C to python is very easy.

3. Open source : No need to pay for python and can be freely downloaded.
   → It's source code can be read, modified and can be used in program as desired by programmer.

4. High level language : Programming languages can be of 2 types :
   1. low-level : Uses machine code instructions and these interact directly

VISION

@python.hub

with CPU.
Ex: machine language and assembly language.

2. High level: Use English words to develop program which is easy to learn & read.

5. Dynamically typed: In python, we need not to declare anything.
An assignment statement binds a name to an object, and object can be of any type
→ That's why python is dynamically typed.
→ Language like c and JAVA are statically typed.

6. Platform independent: When python program is compiled using python compiler, it generates a byte code.
→ Python's bytecode represents a fixed set of instructions that run on all OS. and hardware.
→ Using python virtual Machine (PVM), this byte code can be run on any OS.
→ Hence python not dependent on specefic OS.

7. Portable: When program gives same result on any computer in the world, then it is called portable
→ And as python is platform independent, it will give same result on any computer system.

VISION

@python.hub

8 Procedure & object-oriented: Python is procedure oriented as well as object oriented programming language.

→ Object: Anything that exists physically in the real world.

Let's take student with name Arjun. Object will have behaviour represented by attributes like name, address, contact no. While actions performed by Arjun like eating, sleeping working are represented by method's / function.

Student has attributes:		Arjun has attributes:
name	→	name
address		address
contact no.		contact no.
Student has actions		Arjun has actions:
eating		eating
sleeping		sleeping
working		working

→ Since, class does not exists physically, no memory allocated when class is created. But objects exists physicaly, hence block of memory is allocated when object is created.

vision

@python.hub

9. Interpreted: A python compilers converts source code to byte code. Later this byte code is executed by PVM. Inside PVM, an interpreter converts byte code to machine code line by line so the processor understands it and run to produce results.

10. Extensible: The programs written in C/C++ can be integrated in to python and executed using PVM.
There are different flavours of python wher programs of other languages can be integrated to python.
Ex: Jython, Ironpython

11. Embedded: We can insert python programs into C/C++ programs.
→ Several applications are already developed in python for this purpose

12. Huge Library: Python has a big library which can be used on any OS.
Programmers can easily use modules of python library.

13. Scripting Language: A scripting language is a programming language that does not use compiler for executing source code.
→ Rather it uses an interpreter for direct execution.

VISION

@python.hub

14. Database Connectivity: Python provides interfaces to all major databases like Oracle, Mysql etc.

15. Scalable: A program would be scalable if it could be moved to another os/hardware and take full advantage of new enviornment in term of performance.

16. Batteries included: Huge library of python contains several small applications (packages) which are already developed and immediately available to programmers.
These packages can be used and maintained easily & no need to download seperately. These libraries are called "batteries included"

FLAVORS OF PYTHON

Flavors of python refers to different types of python compilers. These flavors are useful to integrate various programming languages in to python

1. Cpython: This compiler implemented in c language.
   → Internally, python program converted to byte code using c language functions which will run on interpreter in PVM created in c language.
   → It is possible to execute c/c++ functions & programs in Cpython.

vision

@python.hub

2. Jython: Implementation of python programming language which is designed to runs on JAVA platform.
   → Jython compiler first compiles the python code into JAVA byte code and is executed by JVM to give output.
3. Iron Python: Implementation of python language with .NET framework. (written in C# language)
4. PyPy: PyPy is written in Rpython language which was created in python language.
   → pypy programs runs very fast.
5. Ruby Python: This bridges the gap between Ruby and python interpreter.
6. Pythonxy: This is pronounced as python xy and written as python (x,y).
   → Python implementation that we get after adding scientific & engineering related packag
7. Anaconda Python: Python redeveloped for handling large scale data processing, predictive analysis and scientific computing.

FROZEN BINARIES

→ When S/W (sofware) developed in python, two ways to provide it to end users.

1. To provide .pyc file to user.
   The user will install PVM in his computer and run .pyc file.
2. To provide .pyc file along with necessary python library.

VISION

@python.hub

→ In this method, all .pyc files, related python library and PVM will be converted into single executable file (.exe extension) so that user directly execute that file by double clicking it.

→ In this way, converting python program into true executables is called frozen binaries.

→ For creating frozen binaries, developer need to user other party S/W.

For example,

py2exe : it produces binaries for windows

pyinstaller : create frozen binaries for LINUX / UNIX OS.

VISION

@python.hub

Comparision between C and Python.

C	Python.
→ It is procedure oriented. Does not supported classes, objects inheritance, polymorphism etc.	→ It supports object oriented a well as procedure oriented and features like class, objects, inheritance etc
→ C programs executes faster	→ Executes slower compared to PyPy run bit faster but still slower.
→ Compulsory to declare datatypes	→ Type declaration is not requi
→ Statically typed programming language	→ Dynamically typed programming language
→ Pointer concept is available	→ Python does not use point
→ Does not have exception handling facility.	→ Python handles exceptions
→ Has do... while, while & for loops.	→ Has while & for loops only.
→ It has switch statement	→ Does not have switch staten
→ Variable in for loop does not increment automatically	→ Variable in for loop increments automatically.
→ Programmers allocate run time memory using calloc(), malloc(), free(), realloc()	→ Memory allocation and deallocation is automatica done by PVM.
→ Does not support garbage collector	→ An automatic garbage coll is available in python.
→ Supports single & multidimensio -nal array.	→ Supports only single dimensi array, to work with multi- dimensions, need to use num
→ Array index only positive	→ Can be positive / negative
→ Indentation not necessary	→ Indentation is required to represent block of statem

VISION

@python.hub

Comparison between JAVA and Python

JAVA	Python
→ JAVA programs are verebose, means contains more number of lines.	→ Python programs are concise and compact. A big program can be written using less numbe of lines.
→ Compulsory to declare datatypes	→ Not compulsory to declare datatypes
→ Statically typed language	→ Dynamically typed language
→ Semicolon used to terminate statement and comma to seperate expressions.	→ New line indicates end of statements and semicolon used as an expression seperator.
→ JAVA uses do-while, while for each and for loops.	→ Python has while & for loops.
→ It has switch statement	→ Does not have switch statement
→ Variable in for loop does not increment automatically.	→ Variable in for loop increments automatically.
→ Memory allocation is done automatically by JVM.	→ Memory allocation and deallocat is automatically done by PVM.
→ JAVA supports single & multidimensional array.	→ Supports only single dimension, for multidimension we need to use package :numpy
→ Array index should be positive integer.	→ Array index could be positive or negative integer. Negative index indicates location from end of array.
→ Indentation of statement is not necessary	→ Indentation is required to represent block of statements

VISION

@python.hub

COMMENTS IN PYTHON

1. Single line comments :
starts with hash symbol (#) and useful to mention that the entire line till the end should be treated as comment.
Example :

```
1. #To find average of three numbers.
2. a=10  # stores value 10 into a variable a.
```

2. Multi line comments :
When we want to make several line as comment then we have to enclose the lines into a pair of """ (triple double quotes) or ''' (triple single quotes).
Example :

```
''' This program is to find the grade
    of a student based on marks obtained
        in the respective subjects.'''
```

DOCSTRING

→ The triple single quotes or triple double quotes are not actually for comments but they are regular strings with the exception that they can span multiple lines.

→ That means memory will be allocated to these strings internally. If these strings are not allocated to any variable then they are removed from memory by garbage collector. and hence these can be used as comments.

VISION

@python.hub

→ If these strings are written as first statement in the module, function, class or method then these strings are called documentation strings / docstrings which can be used to create API documentation file from python program.

→ An API documentation file is text / HTML file that contains full description of all features like classes, modules and functions of S/W so that user will be able to understand S/W and use it in proper way.

Example:

```
def avg(x,y):
    '''
    This function takes two
    args and find average'''
    print("Avg:",(x+y)/2)

def message():
    '''
    This function display
    a welcome message'''
    print("welcome")
avg(10,10)
message()
```

→ save file as example.

→ To run program:

command: py example.

O/p:

Avg: 10

Welcome

→ Command to generate API documentation file

```
> py -m pydoc -w ex
```

O/p:

~~stir~~

Avg: 10

Welcome

wrote ex.html.

↑

File generated in the same directory.

vision

@python.hub

INDENTATION IN PYTHON

→ Most of programming languages like C, C++ uses braces {} to define a block of code. Python uses indentation.

→ A code of block starts with indentation and ends with first unindicted lines.

→ The amount of indentation can be decided by the programmer, but it must be consistent throughout the block.

→ Generally, 4 white spaces are used for indentation and is prefered over tabspace.

Example :

```
if True:
    print("Yes")
else:
    print("No")
```

✓

Example.

```
if True:
print("Yes")
```

✗

→ python will give you error if you skip indentation.

Example

```
if True:
    print("Yes")
        print("correct")
```

✗

→ You have to use same number of spaces in the same block of code, otherwise it will give you an error

vision

@python.hub

IDENTIFIERS

Example : x=10 # variable name.
def fun() # function name

Rules to define identifier :

1. It can contain both alphabets symbol (upper & lower case), digits & underscore, it is case sensitive
2. It must start with alphabet.
3. Keywords are not allowed as an identifier name
4. There is no length limit of python identifier name.

Note :

1. Identifier starts with _ is private
   Ex : _p
2. Identifier with double underscore at start (__) known as strongly private Ex : (__p)
3. And identifier like __main__ is language specific identifier.

KEYWORDS

True	False	None	and	or	not	is
if	else	elif	while	for while	break	continue
return	in	yield	try	except	finally	assert
raise	import	from	as	class	def	pass
global	nonlocal	lambda	del	with		

→ To display all keywords : run following commands

```
import keyword.
keyword.kwlist.
```

vision

@python.hub

OUTPUT STATEMENT : (print())

→ To display the output we use print() function. This function can be used in different formats :

Type 1 : print()

It will throw the cursor to the next line (blank line is displayed).

Type 2 : print("string")

When a string is passed through ~~display~~ print function, the string will be displayed as it is.

Ex : 1	Ex : 2 Print with escape sequence
print ("Hello")	print ("Hello \n World")
o/p : Hello	o/p : Hello World.

Ex : 3 We can use string repetation character to repeat the string in output as :

```
print ("Hello" * 3)
o/p : HelloHelloHello
```

Ex : 4 We can use string concatenate character (+) to concat two strings

```
print ("city name =" + "jaipur")
o/p : city name=jaipur
```

→ No space after '='

Ex : 5

```
print ("city name =", "surat")
o/p : city name = surat
```

→ Space after '='

VISION

@python.hub

Type 3: print (variable list):

```
Ex: print ("Hello")
    print ("World")
    print ("How are you?")
```

→

```
o/p: Hello
     World
     How are you.
```

→ Here each print function shows cursor to next line

→ "end" attribute can be used to specify ending character of line

```
Ex:
print ("Hello", end='t')
print ("Dear", end='\t')
print ("How are you?", end='')
```

→

```
o/p:
Hello Dear How are you?
```

Type 4: print (object): We can pass objects like lists tuples or dictionaries to print functions.

```
Ex:
l = [10, 20, 30, 40]
print (l)
```

→

```
o/p:
[10, 20, 30, 40]
```

Type 5: print ("string", variable list).

→ Most common use of print is to display string along with variables.

```
Ex:
a = 2
print (a, "is even number")
print ("You typed", a, "as input")
```

→

```
o/p:
2 is even number
You typed 2 as input
```

Type 6: print (formatted string):

→ the output displayed can be formatted as.

Syntax: print ("formatted string" % (variable list))

vision

@python.hub

→ %i or %d to represent decimal integer, %f for float
%s for strings.

```
Ex:
x = 50
print ('x: %i' %x)
```

→

```
o/p:
x: 50
```

```
Ex:
x,y = 10,20
print ('x = %i  y = %d' %(x,y))
```

→

```
o/p
x = 10 y = 20
```

Type 7: print (formatted string with replacement fields):
→ Inside formatted string we can use replacement field denoted by {} contains names/indexes represent order of values.

```
Syntax: print ('formatted string with replacement fields'.
                format (values))
```

```
Ex:
n, o, p = 1, 2, 3
print ('number 1={0}'. format (n))
```

→

```
o/p:
number 1 = 1
```

```
Ex:
n, o, p = 1, 2, 3
print ('n1 = {1}, n2 = {0}, n3 = {2}'. format (o, n, p))
```

```
o/p:
n1 = 1, n2 = 2, n3 = 3.
```

VISION

@python.hub

INPUT STATEMENT : input()

input() :—

→ To accept input from keyboard, python provides the input function. This function takes a value from keyboard and returen it as a string.

→ This function can be used in different format :

Type 1: input() :

```
Ex :
s=input() # this will wait until we enter string
print(s)
```

```
o/p :
COMP
COMP.
```

Type 2 : input ("string")

It displays string message for taking input

```
Ex :
s = input ("Enter your name:")
print(s)
```

→

```
o/p :
Enter your name: Arjun
Arjun
```

```
Ex: Taking an integer as input.
s = int (input ("Enter number:"))
print(s)
```

→

```
o/p :
Enter number: 9
9
```

→ For taking float as input, replace int with float

Type 3 : input("string") :- Reading multiple values from keyboard. using for loop and split().

split() : Useful to seperate value, default argument is space.

VISION

@python.hub

Ex : user wants input : 50 60 (splited by space)

```
l = [int(x) for x in input("Enter two no.'s").split()]
```

or

```
x,y = [int(x) for x in input("Enter two no.'s").split()]
```

Ex :

```
l = [int(x) for x in input("Enter 2 no :").split()]
a,b = l
sum = a+b
print("sum =", sum)
```

o/p :

```
Enter 2 no : 5 6
sum = 11
```

Type 4 : Take arithmetic expression as input and evaluate it.

→ eval() function is useful to evaluate arithmetic expression.

```
x = eval("10+20+30")
print(x)
```

→

o/p :

```
60
```

Ex :

```
s = input("Enter expression :")
x = eval(s)
print('x =', x)
```

→

o/p :

```
10+20
x = 30
```

VISION

@python.hub

PYTHON VARIABLES

→ Variables are containers for storing data values.
→ Python has no command to declare variables
→ A variable is created as you assign a value to it

Ex: `x = 5` # of type int
`y = "computer"` # of type string

✱ Get the type:

Ex:
```
x = 5
y = "Computer
print (type(x))
print (type(y))
```
→
```
o/p:
<class 'int'>
<class 'str'>
```

✱ Assigning multiple values:

1. Many values to multiple variables:

```
x, y, z = "Orange", "Mango", "Apple"
print(x, y, z)
```
→
```
o/p:
Orange Mango Apple
```

Note: No. of variables should match no. of values.

2. One value to multiple variables:

```
x = y = z = "Apple"
print (x, y, z)
```
→
```
o/p:
Apple Apple Apple
```

VISION

@python.hub

PYTHON DATATYPES

→ Data type represent the type of data stored into a variable or memory.

→ Data types already available in python language are called built-in data types.

→ Datatypes which are created by programmer are called User-Defined data types.

Built in data types:

1. Numeric Types : int, float, complex
2. Boolean Type : bool
3. Text Type : str
4. Sequence Type : list, tuple, range.
5. Mapping Type : dict
6. Set Type : set, frozenset
7. Binary Type : bytes, bytesarray.

- int :—

– integer number is a number without a decimal point

– 'int' is a class name.

– Ex : a=10,

>> type(a) → This will display <class 'int'> i.e., 'a' is an object of int class.

Ways to represent values using int

1. decimal form : (base 10) allows number generated using 0-9 digits.

Ex : a=8979

VISION

@python.hub

2. Binary form: (base 2) allows numbers generated using 0,1 digits.

Ex: a = 1111 (i.e a is one thousand one hundred eleven)

a = 0b1111

0b indicates writing binary number.

→ print(a) will display decimal value of '1111' i.e '15'.

3. Octal form: (base 8), allows numbers generated using 0-7 digits

Ex a = 0o10 . print(a) → o/p : 8

↑ 'zero followed by small/capital 'o'

4. Hexadecimal form: (base 16).

Allowed digits: 0-15, A-F/a-f

Prefix: 0x/0X

Ex a = 0x10 → print(a) → o/p : 16

Base conversion from 'int'

1. bin() : converts into to binary
2. oct() : Converts any base to oct
3. hex() : Converts any base to hexadecimal

• float :—

→ float datatype represents floating point numbers.

Ex a = 123.456

→ They can also be written in scientific notations.

Ex : 2.5×10^4 as 2.5E4

Ex x = 2.5e4 → print(x) → o/p : 25000.0

vision

@python.hub

- complex :—

→ To represent complex numbers.

→ Format: a+bj (j is compulsory and at end of imaginary part)
a = real part
b = imaginary part followed by "j"

Ex :			o/p :
	x = 2.5 + 20j		
	print (type(x))	→	<class 'complex'>
	print (x.real)		2.5
	print (x.imag)		20.

Note : In complex number, real part can be hex, decimal, octal or binary but imaginary part must be in decimal.

- bool :—

→ bool datatype represents boolean values which can be either True or false.

→ Python internally represents True as 1 and false as 0.

→ "" blank string also represents 'false'.

Ex :		
b = True	→	o/p :
print (type(b))		<class 'bool'>

- str :—

→ Sequence of characters enclosed within either single/ double quotes.

```
s = 'computer' / s = "computer"  (both are same)
print (type(s))
o/p :
<class 'str'>.
```

vision

@python.hub

Ex: To print message as below:

hello "how" are you?

→ We have to use three single quote or three double quotes:

```
print(""" Hello "how" are you? """)
```

MUTABILITY CONCEPT IN PYTHON.

→ There are two types of objects with respect to mutability concept.

1. mutable 2. Immutable.

→ Since, everything in python is an object, every variable holds an object instance.

→ When an object is initiated, it is assigned a unique object id.

Its type is defined at runtime and once set can never change, however its state can be changed if it is mutable.

"A mutable object can be changed after it is created and an immutable object can't"

int, float, bool, str, tuple	→	Immutable
list, set, dict	→	Mutable

vision

@python.hub

Ex :

```
a=10
print(id(a))        # 1706805520
print(type(a))      # <class 'int'>
b=10
print(id(b))        # 1706805520
b=20
print(id(b))        # 1706805680. here address changes
                    because value is changed and as
                    int is immutable.
```

Ex

```
t1=(10,20,30)
print(type(t1))     # <class 'tuple'>
t1[0]=50            # return error bcoz of tuple is immutable
                    # once created we cannot change it's
                    # value
l=[10,20,30]
l[0]=50             # it will assign '50' at index 0 bcoz
                    # of mutable nature
```

TYPE CASTING

→ Conversion of one data type to another data type.
→ Type casting functions for basic data types are :

1. int (value / variable name)
2. float (value / variabe name)
3. complex (value / variable name)
4. bool (value)
5. str (value / variabe name)

VISION

@python.hub

1. int() :- convert to integer object

Ex:

```
int(123.456) = 123      # float to int
int(10+20j)  = error    # complex to int not allowed
int(True)    = 1        # boolean to int
int(False)   = 0
int("10")    = 10       # str to int.
```

Note: It is allowed only in case of string without decimal point and only if string is in decimal representation.

2. float() :- converts to float object.

Ex:

```
float(10)      = 10.0       # int to float
float(10+20j)  = error      # complex to float not allowed
float(True)    = 1.0        # Boolean to float.
float(False)   = 0.0
float("10.5")  = 10.5       # string to float.
float("10")    = 10.0
float("ten")   = error
```

3. complex(arg1, arg2) :- Converts to complex object.

Here arg2 is optional and default value is 0j.

Ex:

```
complex(10)          = 10+0j       # int to complex
complex(10,20)       = 10+20j      # int to complex.
complex(10.5)        = 10.5+0j     # float to complex.
complex(10.5,20.2)   = 10.5+20.2j
complex(True)        = 1+0j        # Boolean to complex.
complex(False,True)  = 1j
complex(False)       = 0j
float("10.5")        = 10.5+0j     # string to complex.
float("ten")         = error.
```

vision

@python.hub

4 bool(): to convert to boolean object

Ex

```
bool(0) = false      # int to bool
bool(10) = True      # for any non zero, bool return True
bool(10+20j) = True  # complex to bodean
bool(0+0j) = false   # zero value, bool retuen false.
bool(0+1j) = True
bool("") = False
bool("Hello") = True
bool("false") = True.
```

5. str() : convert to string object

Ex

```
str(10) = '10'      # int to string
str(10.5) = '10.5'  # float to string
str(True) = 'True'  # boolean to string
str(10+20j) = '(10+20j)'  # complex to string
```

→ Let is continue with remaining data types.

• bytes

→ bytes datatype represents group of byte numbers. just like an array does. A byte number is any positive integer from 0 to 255. Thus cannot store negative numbers

→ bytes is immutable data type :

Ex: x = [10, 20, 30, 40] b = bytes(x) print(b[0], b[2]) print(type(b)) b[0] = 50.	o/p : 10 20 <class 'bytes'> → Will throw error (Type Error):

VISION

@python.hub

- bytearray()

→ It is similar to bytes but only the difference is bytes is immutable while bytearray is mutable hence content of bytearray can be modified.

```
Ex: x=[10,20,30,40]
    b = bytearray(x)
    print(type(b))
    b[0]=12
    print(b[0])
```

→

```
o/p:
<class 'bytearray'>
12
```

Here the modification is succesful without error.

- list :—

→ List is an ordered collection of data objects seperated by commas.

→ List can contain any type of data such as numbers, strings, list, tuple, range etc.

→ Items in a list are seperated by commas and enclosed in square brackets [].

→ Any object which can be altered after it is created is called mutable object.

→ List is mutable i.e., we can update/add/delete after its creation.

List Creation:

```
l = []          # empty list
l= range(1,20)  # creating a list using range.
l= list()       # creating an empty list using
                  list() constructer.
```

VISION

@python.hub

Ex: l = [1,2,3,4,5]
l = [1, "POOJA", "O", "K", 1, 0, 2]
l = [1, True, False, "P", "Q"]

Accessing list values :

1. Indexing :-

```
l = [1,2,3,4,5]
print (l[0])
print (l[-1])
```

o/p: 1
5

2. slicing :

```
l = [1,2,3,4,5]
print(l[1:2])
print(l[1:])
print(l[:5])
```

o/p: [2]
[2,3,4,5,6]
[1,2,3,4,5,6]

3. Updating list value:

```
l = [1,2,3,4,5]
l[1] = 6
l[3] = 7
l[-1] = 9
print(l)
```

o/p: [1,6,3,7,9]

4. Deleting list element

```
l = [1,2,3,4,5,6]
del l[3]  #delete the index value
print(l)
```

o/p: [1,2,3,5,6]

Methods in LIST :

1. list.append(object): Append an object at the end of the list

```
Ex: l = [1,2,3,4,5]
l.append(6)
l.append("Python")
print(l)
```

o/p: [1,2,3,4,5,6,"Python"]

VISION

@python.hub

2. list.count(object): count the occurrences of specified object.

```
Ex: l = [1,2,3,1,5,1]                    o/p : 3
    p = l.count(1)
    print(p)
```

3. list.remove(object): Remove/delete specified object from list.

```
Ex: l = [1,2,3,4,5,6,7]                  o/p : [1,2,3,5,6,7]
    l.remove(4)
    print(l)
```

4. list.index(object): Return index value of object in list.

```
Ex: l = [1,2,3,4,5,6]                    o/p : 3
    k = l.index(4)
    print(k)
```

5. list.extend(seq): Append the content in sequence to a list.

```
Ex: l = [1,2,3,4]                        o/p : [1,2,3,4,5,6]
    l.extend([5,6])
    print(l)
```

6. list.reverse(): Reverses the list order.

```
Ex: l = [1,2,3]                          o/p : [3,2,1]
    l.reverse()
    print(l)
```

7. list.sort(): Sort elements of list.

```
Ex: l = [5,6,1,4]                        o/p : [1,4,5,6]
    l.sort()
    print(l)
```

8. list.pop(index): pop the specified index value from list.

```
Ex : l = [1,2,3,4,5]                     o/p : [1,2,3,4]
    l.pop(4)
    print(l)
```

VISION

@python.hub

Ex: l=[1,2,3,4,5,6] o/p: [1,2,3,4,5]
l.pop()
print(l)

9. l.insert(index, obj): Insert object at specified index.
Ex: l=[1,2,3,4,5,6,7] o/p: [1,2,3,4,5,"Pooja",6,7]
l.insert(5,"Pooja")
print(l)

10. l.copy(): creates a copy of list with different id.
Ex: l=[1,2,3,4,5,6] o/p: 12456789
l1=l.copy() 19723596
print(id(l))
print(id(l1))

Functions of lists:

1. len(list): return length of list.
Ex: l=[1,2,3,4] o/p: 4
print(len(l))

2. max(list): Return item with maximum value from list.
Ex: l=('s','ss','aaa') o/p: aaa
print(max(l))

3. min(list): Return item with minimum value from list.
Ex: l=(100,200,2,3) o/p: 2
print(min(l))

4 list(seq): Returns lists of any sequence object.
Ex: t=(1,2,3,4,5) o/p: [1,2,3,4,5]
l=list(t)
print(l)

5 sorted(list): Return a list of sorted elements.
Ex: l=[5,4,1,3,2] o/p: [1,2,3,4,5]
print(sorted(l)) [5,4,3,2,1]
print(sorted(l, reverse=True))

VISION

@python.hub

• tuple :-

→ Tuples are immutable ordered sequence of elements.

→ Items in tuple are seperated by commas and enclosed in round brackets.

creation of tuple :

Ex : t=(1,2,3,4)
print(type(t))
o/p : <class 'tuple'>

Ex : t=(1)
print(type(t))
o/p : <class 'int'>

To declare a tuple with one element following syntax is used.

Ex : t=(1,)
print(type(t))
o/p : <class 'tuple'>

Repetition operator with tuple :

Ex : t=(1,2,3)
print(3*t)
o/p : (1,2,3,1,2,3,1,2,3)

Built-in functions of Tuple :

1. len(tuple) : Return length of tuple.
Ex : t=(1,2,3,4) o/p : 4
print(len(t))

2. max(tuple) : Return ~~maxi~~ item with maximum value in tuple
Ex : t=(1,2,3,4,5) o/p : 5
print(max(t))

3 min(tuple) : Returns item with minimum value in tuple.
Ex : t=(100,200,2,3) o/p : 2
print(min(t))

vision

@python.hub

4. tuple(seq): Return tuple of any sequence object.
Ex: l = [1,2,3,4,5] o/p: (1,2,3,4,5)
t = tuple(l)
print(t)

5. sorted(tuple): Sorts elements of tuple into ascending order & return a list of sorted elements
Ex: t = (1200, 147, 2.43, 500) o/p: (2.43, 147, 500, 1200)
print(sorted(t)) [1200, 500, 147, 2.43]
print(sorted(t, reverse=True))

Methods of Tuple:

1. count(x): Returns number of occurrences of element x in tuple.
Ex: t = (1,2,'one',1,5,1) o/p: 3
print(t.count(1))

2. index(x): Returns index of 1st occurrence of element x from tuple.
Ex: t = (1,2,'one',1,5,1) o/p: 0
print(t.index(1))

- range:

→ range represents sequence of values/numbers.
→ It is immutable.
→ There are 3 forms of usage of range datatypes

Form 1: range(10).
It represents value from 0 to (10-1) = 9
Syntax: range(end) = 0 to end-1.
Ex: r = range(10).
→ print(r[0], r[1], r[2] will work but print(r) / slicing

VISION

@python.hub

operator will not work.

Ex:

```
r=range(10)
print(r[0], r[1], r[0:9])
for x in r:
    print(x)
print(r)
r[0]=12.
```

o/p:

```
0 1 range(0,9)
0
1
2
3
4
5
6
7
8
9
rang(0,10).
Type Error:-----
```

Form 2: To represent range not starts from 0.

Syntax: range(start, end) = start to end-1.

Ex: r= range(10,30) # To represent numbers from 10 to 29.

Form 3: To represent range with increment by specific value instead of default value 1.

syntax: range(start, end, stop)

Ex:

```
r= range(10,50,5)
for x in r:
    print(x)
```

o/p:

```
10
15
20
25
30
35
40
45
```

vision

@python.hub

set :

→ Set is a collection which is both unordered and unindexed.

→ First let's see difference between set and list

List	Set
- Insertion order is preserved	Insertion order is not preserved
- Duplicate elements are allowed	Duplication is not allowed.
- Elements are enclosed within square brackets []	Elements are enclosed within {}
→ Indexing and slicing is allowed	Indexing and slicing is not allowed.

Creation of set:

```
Ex: s = {'a', 'b', 'c'}
    print(s)
o/p: {'a', 'b', 'c'}
```

Duplicates not allowed:

```
Ex: s = {'a', 'b', 'c', 'a'}
    print(s)
o/p: {'a', 'b', 'c'}
```

creation of set using set constructor:

```
Ex: s = set(('a', 'b', 'c'))
    print(s)
o/p: {'a', 'b', 'c'}
```

Methods of sets :

1. add() : Adds an element to the set

```
Ex: s = {'a', 'b', 'c'}
    s.add('d')
    print(s)
```

o/p: {'d', 'b', 'a', 'c'}

VISION

@python.hub

2. update() : This method updates the current set, by adding items from another set.

→ If an item present in both set, only one occurrence of that item will be in updated set.

Ex: x = {'a', 'b', 'c'}
y = {'a', 'd'}
x.update(y)
print(x)

o/p: {'a', 'c', 'b', 'd'}

3. pop() : Removes an random element from the set.

Ex: x = {'a', 'b', 'c'}
x.pop()
print(x)

o/p: {'a', 'c'}

4. remove() : Removes the specified element from the set.

Ex: x = {'a', 'b', 'c'}
x.remove('b')
print(x)

o/p: {'a', 'c'}

5. discard() : Remove specified element from set and do not generate error if element doesn't exist, while remove method will raise error if element does not exist.

6. Ex: x = {'a', 'b', 'c'}
x.discard('a')
print(x)

o/p: {'b', 'c'}

Ex: x = {'a', 'b', 'c'}
x.discard('d')
print(x)

o/p: {'a', 'b', 'c'}

@python.hub

Other methods for set

1. union(): Returns a set containing union of set.

Ex: x = {'a', 'b', 'c'} o/p: {'a', 'b', 'c', 'd'}
y = {'c', 'd'}
z = x.union(y)
print(z)

Ex: x = {'a', 'b', 'c'} o/p: {'a', 'b', 'c', 'd', 'e'}
y = {'c', 'd'}
z = {'d', 'e'}
w = x.union(y,z)
print(w)

2. intersection(): Returns a intersection set of two sets.

Ex = x = {'a', 'b', 'c'} o/p: {'c'}
y = {'c', 'd'}
z = x.intersection(y)
print(z)

3. issubset(): Returns whether another set contains this set or not.

Ex: x = {'a', 'b', 'c'} o/p: True.
y = {'f', 'e', 'd', 'c', 'b', 'a'}
z = x.issubset(y)
print(z)

4. issuperset: Returns whether this set contains another set or not.

Ex: x = {'f', 'e', 'd', 'c', 'b', 'a'} o/p: True.
y = {'a', 'b', 'c'}
z = x.issuperset(y)
print(z)

@python.hub

5 isdisjoint(): Returns whether a two sets have a intersection or not

Ex: x = {'a', 'b', 'c'} o/p: True.
y = {'g', 'm', 'f'}
z = x.isdisjoint(y)
print(z)

6 difference(): Returns a set containing the difference between two or more sets.

Ex: x = {'a', 'b', 'c'} o/p: {'b', 'c'}
y = {'g', 'm', 'a'}
z = x.difference(y)
print(z)

7. symmetric_difference(): Returns a set with the symmetric difference of two sets.

Ex: x = {'a', 'b', 'c'} o/p: {'g', 'c', 'b', 'm'}
y = {'g', 'm', 'a'}
z = x.symmetric_difference(y)
print(z)

→ Note:

For any of the set operations you perform and when you print a set, everytime the order of elements in a set are different because set is an ordered sequence.

vision

@python.hub

- frozen set : (set type)

→ frozen set is similar to set only difference is set is mutable while frozen set is immutable.

Ex:

```
s = {10, 20, 30, 40}
fs = frozenset(s)
print(fs)
print(type(fs))
fs.add("hellow")
fs.add("world")
```

o/p:

```
frozenset({40, 10, 20, 30})
<class 'frozenset'>
```

→ Here error will show as it is immutable. (Attribute error :)

- dict : (mapping type)

→ Dictionary represents group of elements arranged in the form of key-value pairs. In the dictionary the first element is considered as a 'key' and the immediate next element is taken as its 'value'.
The key and it's value are seperated by colon(:).

→ All the key value pairs in dictionary are ~~inserted~~ enclosed in curly braces {}.

→ Dictionary is a mutable datatype.

Ex: dictionary 'dict' contain student details.

```
dict = {'Name': 'Vivek', 'Roll_no': 75, 'Std': 7}
```

Here name of the dictionary is dict. It has 3 key value pairs. The first element in dictionary 'Name' which is key and its value seperated by colon is 'Vivek'.

VISION

@python.hub

Some operations on dictionary:

```
d1 = {'name': 'abcd', 'age': 20, 'height': 160}
```

- We can modify existing value of key by assigning a new value as below:

```
d1['age'] = 18
```

- We can insert a new key-value pair to existing dictionary as below:

```
d1['Dept'] = "Computer"
```

Now if we execute print(d1)

```
o/p: {'name':'abcd', 'age': 18, 'height': 160, 'Dept': 'Computer
```

- To delete a key-value pair from dictionary we can use del statement.

```
del d1['height']
print(d1)
o/p: {'name': 'abcd', 'age': 18, 'Dept': 'computer'}
```

- To check whether key is available in dictionary or not we can use 'in' and 'not in' membership operator.

Ex:
```
'Dept' in d1      # return True
'Gender' in d1    # return False.
```

BUILT IN FUNCTIONS OF DICTIONARY

1. len(dict): Returns length of dictionary.

```
Ex: d1 = {'name': 'abcd', 'age': 20, 'Dept': 'computer'}
    print(len(d1))
o/p: 3
```

2. str(dict): Produces printable string representation of dictionary.

VISION

@python.hub

Ex: d1 = {'name': 'abcd', 'age': 20}
print(d1)
print('typecasting to string:', str(d1))
o/p: {'name': 'abcd', 'age': 20}
typecasting to string: {'name': 'abcd', 'age': 20}

3. type(dict): Return type of object passed as an argument. Can be applied to every built-in data types.
Ex d1 = {'name': 'abcd'} o/p: <class 'dict'>.
print(type(d1))

METHODS OF DICTIONARY

1. dict.clear(): Removes all elements of dictionary.
Ex: d1 = {'name': 'abcd', 'age': 20}
print(d1)
d1.clear()
print(d1)
o/p: {'name': 'abcd', 'age': 20}
{}

2. dict.copy(): Returns the copy of dictionary dict.
Ex: d1 = {'name': 'abcd', 'age': 20}
print("d1 before copy:", d1)
print("id:", id(d1))
d2 = d1.copy()
print("d2 after copy:", d2)
print(id(d2))

o/p:
d1 before copy: {'name': 'abcd', 'age': 20}
id: 52229776
d2 after copy: {'name': 'abcd', 'age': 20}
54895600

Two different id's.

@python.hub

3. dict.keys(): Returns a list of keys in dictionary dict.

Ex: d1 = {'name': 'abcd', 'age': 20}
print(d1.keys())

o/p: dict_keys(['name', 'age'])

4. dict.values(): Returns a list of values available in dictionary dict.

Ex: d1 = {'name': 'abcd', 'age': 20}
print(d1.values)

o/p: dict_values(['abcd', 20])

5 dict.items(): Returns a list of dictionary dict's (key, value) tuple pair.

Ex:
d1 = {'name': 'abcd', 'age': 20}
print(d1.items())

o/p: dict_items([('name', 'abcd'), ('age', 20)])

6 dict1.update(dict2): The dictionary dict2's key-value pair will be updated in dictionary dict1.

Ex:
d1 = {'name': 'abcd', 'age': 20}
d2 = {'course': 'BE'}
d1.update(d2)
print(d1)

o/p: {'name': 'abcd', 'age': 20, 'course': 'BE'}

7 dict.get(key, default = None) - Returns the value corresponding to the key specified and if the key is not in dictionary, it returns the default value (or specified default value).

vision

@python.hub

Ex: d1 = {'name': 'abcd', 'age': 20}
print(d1.get('age'))
print(d1.get('phone'))
print(d1.get('phone', 12345))

O/p: 20
None
12345.

8. dict.setdefault(key, default=None) : It will set the key value passed and if the key is not in dictionary, it will set with default value.

Ex:
d1 = {'name': 'abcd', 'age': 20}
print("set key value of (age) in dictionary d1 =", d1.setdefault('age'))
print('set default key value of (phone) in dictionary d1=', d1.setdefault('phone'))

O/p: Set key value of (age) in dictionary d1=20
Set default key value of (phone) in dictionary d1=None

9. d1.fromkeys(seq, [val]): Create a new dictionary from sequence seq and set values from val.

Ex:
l1 = ['Name', 'Age']
d1 = dict.fromkeys(l1)
d2 = dict.fromkeys(l1, ['abcd', 20])
print(d1)
print(d2)

O/p: {'Name': None, 'Age': None}
{'Name': 'abcd', 'Age': 20}.

VISION

@python.hub

10. dict.pop(k[,v]) Removes key 'k' and returns it's value. If key not found then value 'v' is returned. If value v is not specified and key not found then raise exception 'Key Error'.

```
Ex: d1 = {'name' : 'abcd' , 'age':20}
    print ( d1.pop('name'))
    print(d1)
    print(d1.pop('dept', None))
    print (d1.pop('dept'))
o/p: abcd
     {'age':20}
     None
     Key Error :
```

Error for this line.

VISION

@python.hub

Operator.	Description	Example.
Modulus (%)	Divides left hand operand by right hand operand and returns remainder	b % a = 0
Exponent (**)	Performs exponential (power) calculation on operators	a**b = 10 to the power 20 10 **-2 = 0.
//	Floor Division : The division of operand where the result is the quotient in which digits after the decimal point are removed. But if one of the operands is negative, result is floored i.e., rounded away from zero	9//2 = 4 9.0//2.0 = 4.0 -11//3 = -4 -11.0/3 = -4.

→ In floor division if both operands are int then result will be int and if any one or both arguments are float then result is float. i.e., 10/3 = 3.3333 , 10//3 = 3 and 10.0 //3 = 3.0

→ Remark : 1 Use of '+' and '*' operators with string operands :

'+' : string concatenation operator :

Ex : 'ssasit' + '3' => 'ssasit3'

'ssasit' + str(3) => 'ssasit3' # for concatenation both args must be string

'*' : string Repetition / multiplication operator :

Syntax : integernumber * 'string'

OR

'string' * integernumber

→ order not important

Ex : 'ssasit' * 3 => ssasitssasitssasit

@python.hub

OPERATORS IN PYTHON:

→ An operator is a symbol that performs an operation. An operator acts on some variables called operands.

→ For example, if we write a+b, then a and b are operands and + is operator.

→ If operator acts on a single variable than it is called unary operator and if operators acts on two variables than it is called binary operator.

→ Operator in python classified in following categories:

1. Arithmetic Operators
2. Assignment Operators
3. Unary Minus Operator
4. Relational / Comparision Operator.
5. Logical Operator.
6. Bitwise Operator.
7. Boolean operator
8. Membership Operator
9. Identity Operator
10. Ternary Operator.

1. ARITHMETIC OPERATORS

→ There are 7 operators available in python. Arithmetic operators are binary operators

Assume, a=10, b=20 then.

Operator	Description	Example.
Addition (+)	Adds values on either side of operator	a+b=30
Subtraction(-)	Subtracts right hand operator from left hand operator.	a-b=-10
Multiplication (*)	Multiples values on either side of the operator.	a*b=200
Division (/)	Divides left hand operand by right hand operand.	b/a=2

VISION

@python.hub

- 'ssasit' * 3.0 ⇒ Error # only integer numbers are allowed for repitition

Remark 2:
In case of division operation
1. 1/0 : int division by zero error
2. 1.0/0 : float division by zero error.

2. ASSIGNMENT OPERATOR

→ Assignment operator is used to store right side value to left side variable.

→ Assume variable a holds 10 and variable b holds 20 then

Operator	Description	Example.
=	Assigns values from right side operand to left side operand	c = a+b
+= Add	It adds right operand to the left operand and assign the result to left operand	c += a is equivale to c = c+a
-=	It subtracts right operand from left operand and assigns result to left operand	c -= a equivale to c = c-a
*=	Multiplies right operand with left operand and assign result to left operand	c *= a equivalen to c = c*a.
/=	It divides left operands with right operand and assign result to left operand	c /= a equivalent to c = c/a
%=	It takes modulus using two operands and assign result to left operand	c %= a equivalen to c = c%a
**=	Performs exponential calculation on operand and assign result to left operand	c **= a is ≅ c = c** a
//=	Perforns floor division on operands and assign result to left operand	c //= a equivalen to c = c//a.

@python.hub

Notes:

1. In python it is possible to assign same values to more than one variable in the same statement.
   - form 1: Ex: a=b=1
   Form 2: a,b = 1,2.
   ↓ It will be interpreted internally as: a,b = (1,2) and after tuple unpacking it will assign a=1 and b=2.
2. Python does not have increment (++) and decrement (--) operator.
3. If x=100
   if we write, --x then it will consider multiplication of (-,-) which is ('+') so, --x = --10 & ---x = -100
4. a=4
   a&=5 (here & is used as bitwise)
   print(a) => #4

   4 : 100
   (AND) 5 : 101
   ↳ 100 = 4
5. swapping without 3rd variable,
   a,b = 10,20.
   we can swap them as below
   a,b = b,a

3 UNARY MINUS OPERATOR

→ Unary minus operator is denoted by symbol minus (-), when this operator is used before a variable, its value is negated

Ex:
n=10
print(-n) #-10

Ex:
n=-10
print(-n) #10.

vision

@python.hub

4 RELATIONAL / COMPARISION OPERATORS

→ Relational operators are used to compare two quantities.

→ Assume variable a holds 10 and variable b holds 20, then a=10 b=20

Operator	Description	Example.
==	If the value of two operands are equal, then the condition becomes true otherwise false	(a==b) is false.
!= or <>	If values of two operands are not equal, then condition becomes true	(a!=b) is true (a<>b) is true
>	If the value of left operand is greater than the value of right operand then condition becomes true, or else False.	(a>b) is false
<	If the value of left operand is less than the value of right operand, then condition becomes true or else False	(a<b) is True
>=	If the value of left operand is greater or equal to value of right operand then condition becomes true or else False	(a>=b) is Fals
<=	If the value of left operand is less or equal to value of right operand then condition is True or else False	(a<=b) is Tru

vision

@python.hub

Ex-1:

```
a, b = 10, 20                                  o/p:
print("a>b is", a>b)   # False                 a>b is False
print("a>=b is", a>=b) # False                 a>=b is False
print("a<b is", a<b)   # True                  a<b is True
print("a<=b is", a<=b) # True.                 a<=b is True
```

Ex-2 We can apply relational operator to str type also.

```
a, b = "shree", "swami"                        o/p:
print("a>b is", a>b)   # false                 a>b is False
print("a<b is", a<b)   # True.                 a<b is True.
```

→ If first character of both string is same. then 2nd character will be compared and so on.

Ex:3 Chaining of Relational operators.

10 < 20 < 30 < 40 < 50. → If every comparision is True then result is True or else False.

Eg: 10 < 20 > 15 # True.

Ex:4

```
False == False                  # True
10 == 11                        # False
"comp" == "comp"                # True
10 == 20 == 30 == 40            # False
10 == 5+5 == 3+7 == 5*2         # True
(10+2j) == (10+2j)              # True
10 == 10.0                      # True
10.10 == 10.1                   # True.
```

vision

@python.hub

5. LOGICAL OPERATOR.

→ Logical operator are useful to generate compound conditions (Combination of more than 1 simple condition)

→ In case of Logical operator false indicates 0 and True indicates any other number.

Consider, a=10 and b=20

Operator	Description	Example.
and Logical AND	If both operands are true then condition becomes true. In case of non bool (x and y) operand if x is false then result is x else result is y	c = a and b assigns value 'b' to c
or Logical OR	If any of two operands are non zero then condition true. In case of non bool operands (x or y) if x is True then result is x else y	c = a or b Assigns 'a' to c.
not Logical NOT	Used to reverse the Logical state of its operand.	not a → false.

6. BITWISE OPERATOR

→ Bitwise operator acts on individual bits (0 and 1) of operands. We can use bitwise operator directly on binary numbers or on integers also.

→ In case of integers, numbers are converted to binary first and operations are applied on it later.

Let a = 60 (0011 1100)

b = 13 (0000 1101)

VISION

@python.hub

operater	description	Example
& Bitwise AND	If both bits are 1 then result is 1 or else result is 0.	0011 1100 0000 1101 0000 1100
| Bitwise OR	From two operands if any one contains 1 then result is 1 or else 0	0011 1100 0000 1101 0011 1101
^ Bitwise XOR.	If both bits are 1 then result 0 or else any case result is 1	0011 1100 0000 1101 0011 0001
~ Bitwise ones Complement.	Returns one's complement	(~a) = -61.
<< Bitwise left Shift	Left operand value is moved left by the number of bits specified by the right operand.	a<<2 = 240.
>> Bitwise Right shift	The left operand value is moved right by the number of bits specified by right operand.	a>>2 = 15.

7 BOOLEAN OPERATOR.

Boolean literals acts on 'bool' literals. and provide bool type output.

Let a = True , b = false

Operator	Description	Example
and	Both operands True then True or else false.	a and b → False

VISION

@python.hub

or	If any of two operands is True then result True else False	a or b → True
not	Reverse logic state of its operand	not a → False.

IMP & MEMBERSHIP OPERATOR :

Python's membership operator tests for membership in a sequence such as strings, lists, tuples, range, bytes, ... l = [10, 20, 30, 40]

Two membership operator :

1. in 2. not in.

operator	Description	Example
in	Evaluates to true if it finds a variable/value in the specified sequence and false otherwise	a = x in y Here "in" result in a 1 if x is a member of y 10 in l → True
not in	Evaluates to true if it does not find a variable in the specified sequence and false otherwise	a = x not in y here "not in" result a 1 if x not a member of y 10 not in l → False

VISION

@python.hub

9 IDENTITY OPERATOR

→ Identity operator compare the memory location of two objects. There are two identity operators explained below.
Ex a=10 , b=10

Operator	Description	Example.
is	Evaluates to true if the variables on either side of the operator point to the same object and false otherwise.	a is b, here is results in 1 if id(a) = id(b) id(a) = 1926808972 id(b) = 1926808972 So result is True
is not	Evaluates to False if variables on either side of operator point to same object and true otherwise.	a is not b, here is not results in 1 if. id(a) ≠ id(b) id(a) = 1926809872 id(b) = 1926809872 So result is False

10 TERNARY OPERATOR

→ Ternary Operator also known as conditional expressions are operators that evaluates something based on a condition being True or False.

syntax : x = [on_true] if [expression] else [on_false]
Ex : x = "computer" if 1<2 else "comp"
Here condition True , so x = "computer".

sion

@python.hub

Ex:

```
a, b, c, = 10, 20, 30
max = a if a>b>C else b if b>c else c
max = a if a>b and a>c else b if b>c else c
print(max)
```

o/p: 30.

SLICING IN PYTHON

→ First talk about strings:

Syntax: string_variable_name [begin : end : step]

- It returns substring from begin to end-1 index. Here third argument step is optional it is for increment / decrement.
- If value of step argument is not given then default value of increment will be considered as '1'.

```
S = "UNIVERSITY"
S[2:8]   → print character from 2 to 7 index
o/p : 'IVERSI'
```

Positive index: Increases from left to right
Negative index: Expand from right to left

@python.hub

S = "Leave Sorrow"

0	1	2	3	4	5	6	7	8	9	10	11
L	e	a	v	e		S	o	r	r	o	w
-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

[-12 : -5]

S[1:] = eave sorrow.
S[:4] = Leav
S[:] = Leave Sorrow.
S[-1:-4] = No path from -1 to -4
S[-1:-4:-1] = wor.
S[1:10:2] = evor

☆ Similar with list and tuple.

l = [1,2,3,4,5]
l[:3] = [1,2,3]
l[3:] = [4,5]
l[:] = [1,2,3,4,5]
l[::-1] = [5,4,3,2,1]
l[::-2] = [5,3,1]
l[:5:2] = [1,3,5]
l[len(t)::-3] = [5,2]
l[1:4] = [2,3,4]

Similar slicing can be applied to tuple.

vision

@python.hub

CONTROL STATEMENTS IN PYTHON

Control statements are statements which control flow of execution. Python have following control statements.

1. if statement
2. if... else statement.
3. if .. elif .. else statement
4. while loop
5. for loop
6. else suite
7. break statement
8. continue statement
9. pass statement
10. assert statement
11. return statement.

1. if statement :

```
Syntax :
if condition :
    statement(s)
```

if statement is used to execute one or more statements depending on whether a condition is True or not.

→ If condition is True, then statement inside if i.e., after (colon) is executed. we can write one or more statements after (:). If ~~stat~~ condition is False, statement inside if are not executed.

@python.hub

Ex: Python program to check if value is 1.

```
n = 1
if n == 1:
    print ("Yes")
```

o/p:
Yes

Note: Group of statements in python is known as "suite".
We have to take care of proper indentation while writing suite.

Ex: suite using if.

```
X = "comp"
if x == "comp":
    print (" x is comp")
    print ("Yes it is True")
```

o/p:
x is comp
Yes it is True.

2 if ... else statement:

```
Syntax:
if condition:
    statement 1
else:
    statement 2.
```

If else statement is used to a group of statements to be executed when condition is True or else execute another group of statements.

Ex: Python program to check if a given number is even or odd.

vision

@python.hub

```
n= int (input ("Enter a number:"))
if n%2 ==0 :
    print(" Even number")
else :
    print ("odd number")
```

o/p : Enter a number : 2
Even number.

3. if... elif ... else statement.

Syntax:

```
if condition 1:
    statement 1
elif condition 2:
    statement 2
elif condition 3:
    statement 3
else:
    statement 4
```

It is used when we have multiple condition to check and execute statements based on those condition.

→ Condition 1 is True, statement 1 will be executed.

If Condition 1 is False, then condition 2 is checked. if it is true then statement 2 is executed or else condition 3 evaluated.

→ And if none of the conditions are True, then else part i.e., statement 4 is executed.

Ex: Python program to check if a number is 0, +ve, -ve.

```
n = int(input("Enter no:"))
if n >0 :
    print (" +ve)
elif n<0 :
    print("-ve)
else :
    print ("0")
```

o/p :
Enter no: 4
+ve

VISION

@python.hub

4. The nested if statement.

Syntax:

```
if condition 1:
    statement 1
    if condition 2:
        statement2.
    else:
        statement 3
else:
    statement4
```

Ex: Python program to find max out of 3 numbers.

```
x, y, z = 1, 2, 3
if x > y:
    if x > z:
        print("x is max")
    else:
        print("z is max")
else:
    if y > z:
        print("y is max")
    else:
        print("z is max")
```

o/p: z is max

4. while loop:

Syntax:

```
while condition:
    statements.
```

Used to iterate over a block of code as long as condition is True.
Python interprets any non-zero value as True.
None and 0 are as False.

Ex: Python program to add natural numbers from 1 to n

```
n, sum, i = 10, 0, 1
while i <= n:
    sum = sum + i
    i = i + 1
```

vision

@python.hub

```
print('sum:', sum)
```

o/p : Sum : 55.

5. # WAP to print all odd numbers from 10 to 100.

5 for loop :

Syntax :
for variable in sequence :
    statements.

The for loop can be used to execute a group of statements repeatedly depending upon number of elements in the sequence.

→ The for loop can work with string, list, tuple, range, etc.
→ First element of the sequence is assigned to the variable written after 'for' and then statements are executed. Next the second element of the sequence is assigned to the variable and then statements are executed second time.
→ In this way, for each element of sequence, statements are executed once.

Ex : Python program to display characters of string

```
s = "Python"
for i in s :
    print(i)
```

o/p :
P
y
t
h
o
n

vision

@python.hub

Ex: Python program to display characters of string using sequence index.

```
s = "Python"
n = len(s)
for i in range(n):
    print(s[i])
```

o/p:
P
y
t
h
o
n

Ex: Python program to display 10 to 5 in descending order

```
for i in range(10,4,-1):
    print(i)
```

o/p:
10
9
8
7
6
5

Ex: Python program to find sum of list

```
l = [1,2,3,4]
sum = 0
for i in l:
    sum = sum + i
print(sum)
```

o/p: 10

Note: You can also use sum() function to find sum of list/tuple.

vision

@python.hub

- Nested loops :

Ex :

```
for i in range(3):
    for j in range(4):
        print("i =", i, "\t", "j =", j)
```

o/p :

```
i=0  j=0
i=0  j=1
i=0  j=2
i=0  j=3
i=1  j=0
 :
 :
i=2  j=3
```

Ex : To display right angle triangle of *

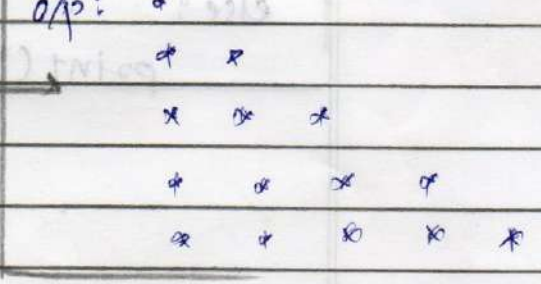
```
for i in range(1,6):
    for j in range(1,i+1):
        print('*', end='')
    print()
```

o/p :

Ex :

```
for i in range(1,6):
    print('*' * (i))
```

o/p :



@python.hub

6 else suite :

→ In python, it is possible to use else statement along with for loop and while loop.

Syntax :

for with else :—	while with else :—

for var in sequence :
  statements
else :
  statements
	while condition :
	  statements
	else :
	  statements.

Ex :

```
for i in range(5):
    print("Yes")
else :
    print ("no")
```

→

```
o/p :
yes
Yes
Yes
Yes
Yes
no
```

Ex :

```
for i in range(0) :
    print ("Yes")
else :
    print ("no")
```

→ o/p : no

vision

@python.hub

Ex: Python program to search element from list of elements.

```
l = [1, 2, 3, 4, 5]
n = int(input("Enter no. to search: "))
for i in l:
    if (i == n):
        print("Element found")
        break.
else:
    print("Not found")
```

o/p 1: Enter no. to search: 4
Element found

o/p 2: Enter no. to search: 20
Not Found.

7. break statement:

→ When break statement is executed, python interpreter jumps out of the loop to process next statement in program

Ex: print numbers from 10 to 6 and break loop when number is 5.

```
x = 10
while x >= 1:
    print("x =", x)
    x -= 1
    if x == 5:
        break
    print("out of loop")
```

o/p:
x = 10
x = 9
x = 8
x = 7
x = 6
out of loop

VISION

@python.hub

8 Continue statement:

→ Whenever continue statement is encountered in loop, it will skip all the lines of loop after continue statement of current iteration and start normal execution of loop from next iteration.

Ex: Python program to print 1 to 5 using continue statement

```
x=0
while x<=10:
    x+=1
    if x>5:
        continue
    print('x=',x)
```

o/p:
x=1
x=2
x=3
x=4
x=5

9 pass statement:

→ 'pass' statement does not do anything.
It is used to represent no operation.
We use pass statement when we need a statement syntactically but we do not want to do any operation.

Ex1: Python program to retreive only negative numbers from list

```
l=[-1,-2,0,1,2]
for i in l:
    if i>=0:
        pass
    else:
        print(i)
```

o/p:
-1
-2

VISION

@python.hub

10 assert statement:

→ 'assert' statement is useful to check if particular condition is fulfilled or not.

syntax: assert expression, message

In above syntax message is not compulsory.

Ex: Python program to ensure that user enter only positive number

```
x = int(input("Enter number:"))
assert x>0, "wrong input"
print('your entered number:', x)
```

o/p: Enter number: 14
Your entered number: 14

Here python interpreter will check if x>0 is True or False.

If True, then next statement after assert will execute, and if False then it will display Assertion Error along with message "wrong input"

o/p: Enter number: -1
Traceback (most recent call last):
File: ....
assert x>0, "wrong input"
Assertion Error: wrong input.

VISION

@python.hub

→ AssertionError in previous example is called Exception. An Exception is an error that occures during runtime. To avoid such type of exceptions, we can handle using try... except statements..

→ After 'try' we can use 'assert' and after 'except' we can write exception name which is going to handle.

Ex: Python program to handle AssertionError exception that is given by assert statement

```
x = int (input ("Enter number:"))
try:
    assert x>0
    print ("U have entered:", x)
except AssertionError:
    print ("wrong input")
```

O/p: Enter number: -1
wrong input

11. return statement:

→ return statement is used inside a function to return some value to the calling place

- Syntax: return expression

Ex: Python program to return a sum of 2 number from a function

```
def sum (a,b):
    return (a+b)
res = sum (10,20)
print ('sum:', res)
```

O/p: sum: 30

vision

@python.hub

As we have see 3 sequence data types list, tuple, range.
Lets get into some deteils into it. and see examples :—

* List : mutable and ordered data type.
→ We have already seen functions & methods of list.

Creating a list() using range function :

```
l = list (range (1, 10, 2))
print(l)
```

o/p : [1, 2, 3, 4, 5, 6, 7, 8, 9]

Updating elements of list :

Method 1 : l.append (10)
print (l)
o/p : [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

Method 2 : Use of slice operator
l[1:3] = 11, 12.
print(l)
o/p : [1, 11, 12, 4, 5, 6, 7, 8, 9, 10]

# Reverse a list.
l.reverse()

Membership in lists :
We can check if element is a member of a list or not by using 'in' or 'not in' operator.
Ex : l = [10, 20, 30, 40, 50]
a = 20
if (a in x): # check if 20 is in X return True
if (a not in x): # return False.

vision

@python.hub

Aliasing of lists :

Giving a new name to existing list is known as 'Aliasing of list. The new name is called 'Alias name'.

Ex: l = [10, 20, 30, 40, 50]
y = l

→ Here we have only one list of elements with two different names 'l' and 'y'.
Here x is original name & 'y' is alias name.

```
Ex: x = [10,20,30]
>>>Y = x
>>> id(x)
    12731856
>>> id(y)
    12731856
```

Address of both 'x' and 'y' are same.

→ If we modify anything in 'x' then changes also reflect in 'y'.

```
x = [10,20,30]
y = x
print(x)
print(y)
x[1] = 23
print(x)
print(y)
```

→

```
o/p:
[10,20,30]
[10,20,30]
[10,23,30]
[10,23,30]
```

VISION

@python.hub

Cloning of list:

If programmer wants two independent list, then he/she should not opt for aliasing, rather go for cloning/copying.

To clone a list 2 methods are there:

1. Slicing operator
2. copy() method.

1. slicing operator:

```
y = x[:]
```

When we clone a list seperate copy of list is created. & 'x' and 'y' are 2 independent lists.

```
Ex: x=[10,20,30]
    y=x[:]
    print(id(x))
    print(id(y))
```

→

```
o/p:
14697936
14699096
```

Two id's are different i.e., cloning is done, two make 2 independent lists.

2. copy() method:

```
Ex: x=[10,20,30]
    y=x.copy()
    print(id(x))
    print(id(y))
```

→

```
o/p:
14697986
14699674
```

Two id's or address of both list are different.

VISION

@python.hub

IMP

List Comprehensions:

→ List comprehension represents creation of new list from an iterable object (like a list, tuple, dictionary or range) that satisfying a given condition.

→ List comprehension contains a very compact code usually a single statement that performs the task.

Ex: Create a list with squares of integers from 1 to 10

```
squares = [ ]
for i in range(1,11):
    squares.append(i**2)
```

Ex: Same example using list comphrension:

```
squares = [x**2 for x in range(1,11)]
```

Some examples:

Ex: Find first letter of every word in a list using list comprehension.

```
l = ['abcd','bcde','cdef']
f = [w[0] for w in l]
print(f)
```

o/p: ['a','b','c']

Ex: Find elements of list present in list1 but not in list2.

```
n1 = [1,2,3,4,5]
n2 = [1,2,5,6,7]
n3 = [X for x in n1 if x not in n2]
print("Element in n1 not in n2 :", n3]
```

o/p: * Element in n1 not in n2 : [3,4]

vision

@python.hub

Ex: Find number of occurrences of a specific element in list and also input elements at run time by user.

```
x = []
n = int(input("How many number of elements ?"))
for i in range(1, n+1):
    i = eval(input('Enter number :'))
    x.append(i)
print(x)
v = int(input("Enter element to count occurrences:"))
c = 0
for i in x:
    if i == v:
        c += 1
print("occurrences of", v, "are", c)
```

o/p:

```
How many number of element? 5
Enter number: 2
Enter number: 2
Enter number: 3
Enter number: 4
Enter number: 5
Enter element to count occurrences: 2
Occurrences of 2 are 2
```

VISION

@python.hub

Ex: Finding common elements of two lists:

```
l1 = ['a', 'b', 'c', 'd']
l2 = ['b', 'c', 'd', 'e']
s1 = set(l1)
s2 = set(l2)
s3 = s1.intersection(s2)
res = list(s3)
print(res)
```

o/p: ['b', 'c']

Here, we have converted list to set so that we can use intersection function of set to easily find common elements.

Ex: Python program to print keys, values, key value list and perform addition of values of dictionary.

```
d1 = {'A':10, 'B':20, 'C':30, 'D':40, 'E':50}
print ("keys :", d1.keys())
print ("values:", d1.values())
print ("List of key values :", d1.items())
s = sum (d1.values())
print ('sum:', s)
```

o/p: keys : dict_keys(['A', 'B', 'C', 'D', 'E'])
values: dict_values([10,20,30,40,50])
List of key values: dict_items([('A',10),('B',20),('C',30), ('D',40), ('E',50)])

sum: 150

vision

@python.hub

Ex: python program to create dictionary with cricket player names and scores in match. Retreive score by entering name.

```
x={}
n=int(input("Enter no. of players:"))
for i in range(n):
        name=input("Enter name:")
        score=int(input("Enter score:"))
        d1={name:score}
        x.update(d1)
print('players:',x.keys())
print('Scores:',x.values())
n1=input("Enter name of player:")
s=x.get(n1,-1)
if s!=-1:
        print('scores:{1} made by {0}'.format(n1,s))
else:
        print('Player not found')
```

o/p:

```
Enter no. of players:2
Enter name:dhoni
Enter score:99
Enter name:sachin
Enter score:100
players: dict_keys(['dhoni','sachin'])
scores: dict_values([99,100])
Enter name of player:sachin
Scores:100 made by sachin
```

@python.hub

Ex: Python program to demonstrate use of for loop to retrieve elements of dictionary.

```
c = {'a': 'Apple', 'b': 'ball'}
for i in c.values():
    print(i)
for i in c.keys():
    print(i)
for i,j in c.items():
    print(i,j)
```

→

```
o/p: a
     b
     Apple
     ball
     a, Apple
     b, ball
```

Ex: Python program to find occurrences of each letter in a string using dictionary.

```
s = 'compputer'
d = {}
for i in s:
    d[i] = d.get(i,0)+1
print(d)
```

```
o/p:
{'c': 1, 'o': 1, 'p': 2,
 'u': 1, 't': 1, 'e': 1, 'r':
```

Ex: Python program to print series of prime numbers

```
m = int(input("Enter number:"))
for i in range(2,max+1):
    count=0
    for j in range(1,i+1):
        if(i%j)==0:
            count+=1
    if(count==2):
        print('\t',i,end='')
```

```
o/p:
Enter number: 11
 2  3  5  7  11
```

VISION

@python.hub

Ex: Python program to print fibonacci series of upto n step.

```
n = int(input("Enter step:"))
f0 = 0
f1 = 1
if (n == 1):
    print('0')
elif n == 2:
    print(f0, '\t', f1)
else:
    print(f0, '\t', f1, end = '\t')
    for i in range(1, n-1):
        f0, f1 = f1, f0+f1
        print(f1, end = '\t')
```

o/p: Enter step: 8
0 1 1 2 3 5 8 13

Ex: Python program to check whether a given no. is Armstrong or not.

```
n = int(input(" Enter 3 digit no.:"))
sum, temp = 0, n
while temp > 0:
    digit = temp % 10
    sum = sum + digit**3
    temp = temp // 10
if sum == n:
    print('armstrong')
else
    print('not armstrong')
```

o/p: Enter 3 digit no: 153
armstrong

@python.hub

Ex: Python program to find the cube root of perfect cube.

```
x = int (input ("Enter perfect cube number:"))
num = 0
print (abs(x))
while  num**3 < abs(x):
        num = num + 1.
print (num)
if  num **3 != abs(x)
        print (x, "is not perfect cube value")
else :
        if (x<0):
            num = -num
        print (x," is perfect cube value of", num).
```

```
o/p : Enter perfect cube number: 27.
      27
      3
      27 is perfect cube value of 3.
```

vision

@python.hub

FUNCTIONS IN PYTHON

→ Function is similar to program that consists of a group of statements that are intended to perform a specific task.

→ Once function is written, it can be reused whenever required.

Difference between function & method:

→ A function can be written individually in the python program. A function is called using its name.

→ When a function is written inside a class, it becomes a 'method'.

A method is called as:

objectname.methodname()

classname.methodname()

Ex:

```
s = "hello"
c = len(s)    # function call
```

Ex:

```
s = "hello"
s1 = s.toupper()    # method call
```

USER-DEFINED FUNCTIONS:

We can define a function using keyword def followed by function name. After function name we should write parenthesis which may contains parameters.

Syntax:

```
def function-name (arg1, arg2 ...):
    '''function_docstring'''
    function statements
```

@python.hub

Ex:

```
def avg(a,b):
    ''' function to find average '''
    c=(a+b)/2
    print(c)
```

→ Here ':' represents beginning of function body.
→ Here arg1, arg2 are the arguments/parameters passed to functions.

function call:

Syntax: function_name(parameters)

→ Function will run only when we call it.
→ The above function can be called:

Ex: avg(10,20) o/p: 15

returning value/Result.

→ We can return a result from a function using return statement.
→ return c
return 100
return [10,20,30]

Ex: Function to test if a given number is even or odd.

```
def check(num):
    if num%2==0:
        print("even")
    else:
        print("odd")
check(20)
check(103)
```

o/p: even
odd

VISION

@python.hub

Ex: function to check given number is prime or not.

```
def prime(n):
    count=0
    for i in range(1,n+1):
        if n%i==0
            count+=1
    if count==2:
        print("prime")
    else:
        print("not prime")
prime(2)
prime(6)
```

o/p: prime
not prime

Ex: function to find factorial of a number

```
def fact(n):
    f=1
    for i in range(1,n+1):
        f=f*i
    print("Factorial ({0}) is {1}".format(n,f))

fact(5)
fact(10)
```

o/p: Factorial (5) is 120
Factorial (10) is 3628800

@python.hub

Ex: Python program to find reverse of a given number using user defined function.

```
def rev_num(n):
    r=0
    while(n!=0):
        d=n%10
        r=r*10+d
        n=n//10
    return r
n=int(input("Enter number:"))
r=rev_num(n)
print("Reverse:",r)
```

o/p: Enter number: 1234
Reverse: 4321

Returning multiple values from function:

→ In python we can return multiple values.
return a,b,c.

→ values returned by a function as a tuple.
To grab these values we can use 3 variable.
x, y, z = function_name()

Ex:

```
def mul_div(a,b):
    c=a*b
    d=a/b
    return c,d
~~mul_div(20,10)~~
x,y=mul_div(20,10)
print(x,y)
```

o/p: 200
2.

vision

@python.hub

Function Arguments:

We can call the function by any of following arguments:
1. Required arguments / positional arguments
2. Keyword arguments
3. Default Arguments
4. Variable length Argument.

1. Required Arguments:

Required Arguments are passed to the function in correct positional order.
Here the number of arguments in a function call should match exactly with the function definition.

Ex:

```
def attach(s1,s2):
    s3=s1+s2
    print(s3)
attach('com','puter')
attach()
```

o/p:

```
Computer
→ Error for this line
(Type Error:....)
missing 2 required
positional arguments)
```

2. Keyword Arguments:

When we use keyword arguments in a function_call, the caller identifies the arguments by parameter name.
This allows us to skip arguments / place them out of order because the python interpreter is able to use keywords provided to match the values with parameters

@python.hub

Ex:

```
def shop (product, cost)
    print ("product:", product)
    print ("cost:", cost)
shop (product = 'sugar', cost = 100)
shop (cost = 85, product = 'flour')
```

```
o/p: Product : sugar
     Cost : 100
     Product : Flour
     Cost : 85
```

3. Default Arguments:

→ Default Argument is an argument that assumes a default value if a value is not provided in the function call for that argument.

→ If value provided of default argument then argument provided in function call is used.

Ex:

```
def shop (product, cost = 100)
    print ("Product:", product)
    print ("Cost:", cost)

shop ('Flour', 200)
shop ('Biscuit')
```

```
o/p : Product : Flour
      Cost : 200
      Product : Biscuit
      Cost : 100.
```

vision

@python.hub

4. Variable length arguments:

→ In some cases we may need to process a function for more arguments then specified while defining a function.

→ These arguments are called variable length arguments and are not named in the function definition, unlike positional and default.

→ A * is placed before a variable name that holds the values of all non keyword variable arguments.

→ This tuple remains empty if no additional arguments are specified during the function call.

Syntax: def function-name (formal-arguments, * variable-tuple):

→ We can pass one or more arguments to this * variable-tuple and it will store all them in a tuple.

Ex: Python function to print sum of all formal arguments and variable length arguments.

```
def add (farg, *args):
    sum=0
    for i in args:
        sum=sum+i
    print ("sum:", farg+sum)
add (5,10)
add (5,10,15,20,30)
```

O/p: sum: 15
sum: 80

@python.hub

IMP

FILTER FUNCTIONS:

→ The filter() function constructs an iterator from elements of an iterable for which a function return True.

→ In simplewords, the filter() function will filter the given iterable with the help of a function that test each element in the iterable to be True.

Syntax: filter(function, sequence).

function: It represents a function name that may either return True or False.
If 'None', the function defaults to identity function which return false, if any elements are 0 or False & True for non-zero elements.

sequence: Iterable/ sequence eg: sets/lists/tuples/strings.

→ The "filter() function will be applied to every element of sequence and when function return 'True', the element is extracted otherwise it is ignored.

1. Filter() working for iterable list:

Ex: filter out even number from list

```
def even(x):
    if x%2==0:
        return True
    else:
        return False
```

vision

@python.hub

```
l = [1,2,3,4,5,6]
n = list(filter(even, l))
print("old list:", l)
print("new list:", n)
```

```
o/p : [2,4,6]
old list: [1,2,3,4,5,6]
new list: [2,4,6]
```

2. filter() method working without function.

```
l = [1, 'a', 0, False, True, '0']
n = filter(None, l)
for i in n:
    print(i)
```

```
o/p : 1
      a
      True
      0
```

@python.hub

MAP FUNCTION :

→ map() function is used to apply a function on all elements of a specified iterable & return map object. We can iterate over map object.

→ map() is similar to filter() but it can act on each element of iterable & can change its value.

Syntax: map (function, iterable, ...)

function : function performs the specified operation on all elements of iterable and modified elements are returned.

iterables : We can pass multiple iterable arguments to map() function. In that case, function should also have that many arguments.
→ With multiple iterable argument, the map iterator stops when the shortest iterable is exhausted.

Consider this function

```
def toupper(s):
    return str(s).upper()
```

1. map() with string.

```
n = map (toupper, 'abc')
for i in new:
    print(i)
```

o/p: A
B
C

vision

@python.hub

2. map() with tuple:

```
n = map (toupper, (1, 'abc', 'a'))
for i in n:
    print(i)
```

o/p: 1
ABC
A

3. map() with list:

```
n = map(toupper, ['x', 'ab', 'abc'])
for i in n:
    print(i)
```

o/p: X
AB
ABC

Ex: Find a square of all elements of list using map.

```
def s(x):
    return x*x
l = [1, 2, 3, 4, 5]
n = list(map(s, l))
for i in n:
    print(i).
```

o/p: 1
4
9
16
25

@python.hub

Ex : map() with multiple arguments

```
def mul(x,y):
    return x*y
l = [1,2,3,4]
m = [5,6,7,8]
n = list(map(mul, l, m))
print(n)
```

o/p : [5, 12, 21, 32]

MIMP

LAMBDA (Anonymous functions)

→ In python ~~anyn~~ anonymous functions is a function that is defined without name.

→ While normal functions are defined using 'def' keyword, anonymous function are defined using 'lambda' keyword.
Hence anonymous functions are also called as lambda functions.

Characterstics of lambda functions :

1. Lambda functions can take any number of arguments but return only one value in the form of expression
2. It cannot contain multiple expression.
3. It cannot have comment.
4. A lambda function cannot be a direct call to print because lambda requires an expression.

VISION

@python.hub

5. Lambda functions have their own local namespace and cannot access variable other than those in their parameter list & those in global namespace.
6. Lambda function is not equivalent to inline function of C++.

Syntax:
lambda argumentlist : expression

Defining a lambda function:
variable_name = lambda argument_list : expression

Calling lambda function:
variable_name(argument_list)

Example:

Program 1: Lambda function to find square of a number
Code :
```
square = lambda x : x * x
print (square(4))
```
O/p : 4

Program 2 : Lambda function to add 2 numbers
Code :
```
add = lambda x,y : x+y
i= int(input (" Enter 1st no.:"))
j= int(input (" Enter 2nd no.:"))
print (add(i,j))
```
O/p :
```
Enter 1st no.: 2
Enter 2nd no : 3
5
```

@python.hub

MIMP

Program 3: lambda inside function

```
Code : def test(n):
           return lambda x: x*n
       result = test(10)
       print(result(9))
```

O/p : 90

Use of lambda function:

We can use lambda function when want a nameless for a short period of time. we generally use it as a argument to higher order function.

✱ Lambda with filter() function:

Program: Lambda function to filter odd numbers

```
Code : l = [1,2,3,4,5,6,7,8,9,10]
       n = list(filter(lambda x: x%2!=0, l))
       print(n)
```

O/p : [1, 3, 5, 7, 9]

✱ Lambda with map() function:

Program: To increment every item of list by 2

```
Code : l = [1,2,3,4,5,6,7]
       n = list(map(lambda x: x+2, l))
       print(n)
```

O/p : [3, 4, 5, 6, 7, 8, 9]

vision

@python.hub

Program : multiply two lists using lambda with map.

Code :
```
l1 = [1,2,3,4]
l2 = [5,6,7,8]
n = list (map (lambda x,y : x*y , l1, l2))
print(n)
```

o/p : [5, 12, 21, 32]

Difference :

filter()	map()
→ filter() function is useful to filter out element of a sequence depending on result of function	→ map() function is similar to filter() but it act on each element of sequence and change its value.
→ Syntax: filter (function, sequence) Here function may be userdefined or lambda function & sequence may be list, tuple or string.	→ Syntax: map (function, sequence(s)) Here function may be userdefined or lambda function and sequence may be list, tuple or string
→ filter function must have only one sequence variable as argument	→ map function may have 1 or more sequence variable of same length.
→ l = [2, 31, 45, 67] l1 = list (filter (lambda x: x%2 != 0, l)) print (l1) o/p: [31, 45, 67]	→ l1 = [1,2,3] l2 = [5,6,7] l = list (map(lambda x,y : x*y, l1, l2)) print (l) o/p: [5, 12, 21]

@python.hub

RECURSIVE FUNCTION

→ Recursive function is a user defined function which calls itself.

Ex: Recursive function to find factorial of a given number.

```
def fact(n):
    if n==1:
        return 1
    else:
        return (n*fact(n-1))
f1=fact(5)    # function call
print("Factorial : ", f1)
```

o/p:
Factorial : 120

Ex: Recursive function to print binary corresponding to decimal.

```
def dec_bin(n):
    if n>=1:
        dec_bin(n//2)
        print(n%2, end=' ')
dec_bin(4)    # function call
```

o/p:
1 0 0

@python.hub

LOCAL AND GLOBAL VARIABLE

→ A variable defined inside a function is called local variable

→ A local variable is a variable whose scope is only limited to that function where it is created.

→ That means the value of local variable is available only in that function and not outside of that function. In below example, 'a' is declared inside a function my_funct(), hence it is a local variable of my_funct(). Once we come out of the function, the variable 'a' is removed from memory.

Ex:

```
def my_funct():
    a=1
    a+=1
    print(a)
my_funct()
print(a)
```

o/p

```
2
Traceback (most ....):
  File ....
  ....
NameError: name 'a' is not defined.
```

→ When a variable declared outside/above function it becomes global variable. such variables are available to all functions written

→ scope of local variable is limited to a function where it is declared only. the scope of global variable is the entire program body written below it.

@python.hub

Ex

```
a=50
def my_funct():
    print('a inside fun',a)
my function()
print('a outside fun',a)
```

o/p:

```
a inside fun 50
a outside fun 50
```

global keyword :-

→ Sometimes local variable and global variable may have same name. In that case, function by default refers to the local variable and ignores the global variable, so the global variable is not accessible inside the function but outside of it is accessible.

→ When programmers wants to use global variable inside function he can use keyword global before variable name on begining of function body.

Ex

```
a = 50
def my_func():
    a = 2
    print('a inside fun', a)
my_func()
print('a outside fun', a)
```

o/p:

```
a inside fun 2
a inside fun 50
```

VISION

@python.hub

accessing global variable inside function

```
a=50
def my_fun()
    global a
    print('global a in fun ', a)
    a+=1
    print('after increment in global variable : ',a)
my_fun()
print('a outside fun ', a)
```

→ In python function, if name of local and global variable are same then globals() function is used to return table of current global variable in form of dictionary.

→ `x= global()['a']` // inside function this statement is used to get global variable to x. after that function will use 'a' as a local variable and x is global

VISION

@python.hub

FILE HANDLING IN PYTHON

→ File is named location on disk to store related information
→ It is used to store permanently data in a non-volatile memory.
→ When we need to read/write data to/from file, we need to open it first, and after completion of operation on file, we need to close it, so that resources that are tied with the file are freed.
→ Python file operations takes place in following order:
1. Open a file.
2. Read/Write (perform action).
3. Close the file.

Opening a file :

→ Before we can read/write a file, we have to open it using python's built in function open().
→ This function returns a file object, also called a handle, as it is used to read/modify the file accordingly
→ We can specify mode while opening a file. In mode we specify whether we want to open file for read 'r', for write 'w', or append 'a'.
→ We can also specify to open a file in text format or binary format. Default reading is in text mode. In this mode we get strings when we read from file.
→ On the other end, binary mode returns bytes and this is the mode to be used when dealing with non-te files like image/exe files.

vision

@python.hub

Opening a file :

Syntax : file-object = open (filename [, access_mode] [, buffering])

Example : f = open ('C:/pythonsy/abc.txt', 'r')

Example : f1 = open ('1.txt', 'w')

filename : The file name arguments is string value that contains name of file that we want to access.

accessmode : This is optional field. Default file access mode is 'r' - read.
The access mode determines the mode in which you want to read/write which can be 'r', 'w', 'r+', 'w+', 'rb', 'wb', 'rb+', 'wb+', 'rb', 'wb', 'a', 'ab', 'a+', 'ab+', 'x'.

buffering : This is optional parameter. If buffering is set to 0, no buffering will take place. If buffering value is 1, line buffering is performed while accessing a file.
If we specify buffering value as an integer greater than 1, then buffering action is performed with indicated buffersize.

vision

@python.hub

MODE OF OPENING A FILE

Mode	Description
r	Open a file in read only mode. The file pointe is placed at beginning of file. This is default mode.
rb	Open a file for reading only in binary format. The file pointer is placed at beginning of file.
r+	Open a file for both reading and writing. The file pointer is placed at begening of file.
rb+	Opens a file for both reading and writing in binary format. The file pointer is placed at begining of file.
W	Opens a file for writing only. Overwrites the file if the files exist. If file does not exist, creates a new file for writing.
wb	Opens a file for writing only in binary format. Overwrite the file if already exist. If file does not exist, creates a new file for writing.
w+	opens a file for both reading and writing. Overwrites the if if already exists. If file does not exist, creates a new file for reading and writing.
wb+	Opens a file for both reading and writing in binary format. Overwrites the file if already exists. If file does not exists, create a new file for both reading and writing.
a	Opens a file for appending. The file pointer is at the end of the file, if exists. If file does not exists then it will create a new file for writing.
ab	Opens a file for appending in binary format. The file pointer is at the end of the file, if file exists. If file does not exists then it will create a new file for writing

VISION

@python.hub

a+ Opens a file for appending and reading. The file pointer is at the end of file if the file exists. If file does not exists then it will create a new file for reading and writing.

ab+ Opens a file for appending and reading in binary format The file pointer is at the end of the file if exists If file does not exists then it will create a new file for reading and writing.

x. Opens the file for exclusive creation. If file already exists, the operation fails.

ATTRIBUTES OF FILE OBJECT

→ Once file is opened, we have one file object, we can get various information related to that file. The file object have following attributes.

1. file.closed : Returns true if file is closed, false otherwise
2. file.mode : Returns access mode with which file was opened
3. file.name : Returns name of file
4. file.softspace : Returns 0 if space is explicitly required with print or 1 otherwise. (Not supported by python 3)

@python.hub

Ex: Program to display attributes of file object.

```
fo = open ('file.txt', 'w')
print ('name of file:', fo.name)
print ('mode of file:', fo.mode)
print ('closed or not:', fo.closed)
```

o/p:
name of file: file.txt
mode of file: w
closed or not: false.

CLOSING A FILE

→ When we have finished operations of file, we need to properly close it.

→ Python has garbage collector to clean up un referenced objects.
But we must not rely on it to close the file, closing a file will free up the resources that were tied with the file and is done using close() method

Syntax: fileobject.close()

Ex: Open a file in binary format

```
fo = open ('f.txt', 'wb')
print ('name:', fo.name)
fo.close()
print ('closed?', fo.closed)
```

o/p
name: f.txt
closed? True.

vision

@python.hub

WRITING TO A FILE

→ In order to write on to file, we need to open it with 'w', append 'a' or exclusive creation 'x' mode.

→ We need to be careful with 'w' mode as it will overwrite file if file is already exists. All previous data will be deleted.

→ The write() method writes anystring to an open file. It is important to note that python strings can have binary data and not just exist.

→ The write() method does not add a newline character ('\n') to the end of string.

Syntax: fileobject.write(string)

Ex: Program to write string to file.

```
fo = open ('f.txt', 'w')
fo.write ("Python In Progremming")
fo.close.
```

→ The above code would create f.txt file and write given content to it and closes the file.

If you open f.txt →

f.txt.

Python
Progremming

vision

@python.hub

The read() Method

→ To read content of file, we must open the file in reading mode.

→ The read() method reads a string from an open file. It is important to note that python strings can have binary data. apart from text data.

Syntax : fileobject.read([size])

Here passed parameter size is the number of bytes to be read from the opened file. This method starts reading from the beginning of the file and if size is missing, then it tries to read as much as possible, maybe until end of the file.

f.txt → Python Programming

Ex: Program to read content of file.

```
fo = open ('f.txt', 'r+')
str= fo.read (10)
print ("string :", str)
```

o/p:
Python Pro

Ex:

```
fo = open ('f.txt', 'r+')
str= fo.read()
print ("string :", str)
fo.close()
```

o/p
Python Programming

@python.hub

FILE METHODS

1. file.close() : To close an open file.

2. file.fileno() : Returns integer number (file descriptor) of file.

3. file.seek(offset, from = SEEK_SET) : Change the file position to offset bytes, in reference from (start, current, end)
   - → The offset argument indicates the number of bytes to be moved.
   - → The from argument specifies the reference position from where the bytes are to be moved.
   - → If from is set to 0, the begining of file is used as the reference position.
     If set to 1, current position is used as reference.
     If set to 2, end of file is taken as reference.

4. file.tell() : Returns the file current position. The tell() method tells you current position within the file.

5. file.write(str) : To write string in opened file.

6. file.read() : To read content of opened file.

7. file.readline() : This method reads one entire line from file. A trailing new character is kept in string. An empty string returns only when EOF is encountered immediately.

@python.hub

8. file.truncate(size): It truncate file size.

9. file.readlines(sizeint): This method reads until EOF using readline() and returns a list containing the lines.
   → sizeint is an optional argument and if it is present, instead of reading upto EOF, whole lines approximate to sizeint bytes are read.
   → An empty string is returned only when EOF encountered immediately

10. file.writelines(sequence): This method write sequence of strings to file. The sequence can be any iterable object containing strings. No Return Value.

Ex: Program to use seek() and tell()

```
fo = open("fo.txt", 'r+')
str = fo.read(10)
print ("String :", str)

pos = fo.tell()
print ("Current position:", pos)
# Repositioning to beginning
pos = fo.seek(0,0)
str = fo.read(10)
print ("String:", str)

fo.close()
```

O/P:
String: Python Pr
Current Position: 10
String: Python Pr

VISION

@python.hub

f.txt
Python Programming

Ex: Program to demonstrate read(), readline(), readlines()

Write a python program to append data in existing file and display.

```
fo= open('f.txt','a+')
print ('Enter text to append (@ to quit)')
s=''
while (s!='@')
    s=input ('Enter string:')
    if (s!='@'):
        f.write(s+'\n')
f.seek(0,0)
print ('Demonstration of read()')
s=f.read()
print(s)
print ('Demonstration of readline()')
f.seek (0,0)
s=f.readline()
print(s)
print ('Demonstration of readlines()')
f.seek(0,0)
s=f.readlines()
print(s)
f.close().
```

VISION

@python.hub

o/p:

```
Enter text to append (@ to quit)
Enter string : I love Python
Enter string : @
Demonstration of read()
Python Programming
I love Python
Demonstration of readline()
Python Programming
Demonstration of readlines()
['Python Programming\n', 'I love Python\n']
```

Ex: Program to use truncate() method

```
fo.open('f.txt','r+')
l1 = fo.readline()
print ('read line 1:', l1)
loc = fo.tell()
print ('file location:', loc)
fo.truncate()
r = fo.readlines()
print ('Remaining lines:', r)
fo.close()
```

f.txt

```
Python Programming
I love Python
Do you?
```

o/p:

```
Python Programming
```

o/p:

```
read line 1: Python Programming
file location: 20
Remaining lines: []
```

After execution:

f.txt

```
Python Programming
```

@python.hub

Ex: Program to demonstrate writelines() method

```
fo = open('f4.txt', 'wt')
seq = ['line 1\n', 'line 2\n', 'line 3\n']
fo.writelines(seq)
fo.seek(0,0)
lines = fo.readlines()
print ("Lines:", lines)
fo.close.
```

After execution:
f4.txt
line 1
line 2
line 3

o/p:
Lines: ['line 1\n', 'line 2\n', 'line 3\n']

KNOWING WHETHER FILE IS EXIST OR NOT

→ The operating system 'os' module has sub by name 'path' that contains a method is file(). this method can be used to know whether a file that we are openeng really exists or not.

Ex: os.path.isfile('f.txt') give True if file exists else False

Ex:

```
fname = input ("Enter file name:")
if os.path.isfile (fname):
    f.open(fname, 'r')
else:
    print ('file does not exist')
```

@python.hub

python.py
print ("Hello")

← python.py file content

Ex: Write a python program to append data to an existing 'python.py'. Read data to be attended from user. Then display the content of entire file.

```
import os
fname = input ('Enter filename to open')
if os.path.isfile (fname):
    f= open(fname,'a+')
else:
    print (fname,"is not exists")
    sys.exit()
print('Enter text to append (@ to quit)')
s = ''
while (s!='@'):
    s=input('Enter string')
    if (s!='@')
        f.write (s+'\n')
f.seek(0,0)
print ("Reading:")
s1= f.read()
print (s1)
f.close()
```

o/p:

```
Enter filename to open python.py
Enter text to append (@ to quit)
Enter string: print("world")
Enter string: print("Hi")
Enter string: @
Reading:
print("Hello")
print("world")
print("Hi")
```

vision

@python.hub

MTMP

Ex: Read a text file in python and print no. of lines and no. of unique words.

Logic: Use for loop to read line by line from file.

```
for line in f:      # repeat loop for all lines
    print(line)     # display one line at a time
    words = line.split()  # words will be stored in
                          a list from line.
```

Code:

```
import os
import sys
name = input("Enter file name:")
if os.path.isfile(fname):
    f = open(fname,'r')
else:
    print('File not exist')
    sys.exit()
cl = cw = cc = 0
l1 = []
for line in f:
    print(line)
    word = line.split()
    print(word)
    l1.extend(word)  # insert list word's element to l1)
    cl += 1          # add line count
    cw += len(word)  # add words count.
    cc += len(line)  # add character count.
print("Total lines:", cl)
print("Total words:", cw)
print("Total characters:", cc)
```

vision

@python.hub

```
print('list of words:', l1)
# to find unique words.
l2=[]
for x in l1:
    if x not in l2:
        l2.append(x)
print('unique words are:',l2)
```

f6.txt:

```
Python is a great language.
Yeah it is great!
```

O/P:

```
Enter file name: f6.txt
Python is a great language.
['Python', 'is', 'a', 'great', 'language.']
Yeah it is great!
['Yeah', 'it', 'is', 'great!']
Total lines : 2
Total words : 9
Total characters 47
list of words : ['Python', 'is', 'a', 'great', 'language.', 'Yeah',
                 'it', 'is', 'great!']
Unique words are: ['Python', 'is', 'a', 'great',
 'language.', 'Yeah', 'it', 'great!']
```

vision

@python.hub

The with statement

→ 'with' statement can be used while opening a file. The advantage of 'with' statement is that it will take care of closing a file which is opened by it. Hence need to close the file explicitly.

→ In case of exception, with statement will close the file before exception is handled.

Syntax: with open ('filename','mode') as file-object:

```
Ex:
with open ('f.txt','w') as f1:
    f1.write ('I am Arjun\n')
    f1.write ('I love Python\n')
with open ('f.txt','r') as f2:
    for line in f2:
        print(line)
```

o/p: I am Arjun
I love Python

@python.hub

WORKING WITH BINARY FILE

→ Binary files handles data in form of bytes. Hence they can be used to read/write, text, image, audio, video, files.

→ Below program example reads data from image file 1.jpg and write its data to new binary file 2.jpg.

```
Ex:
f1 = open ('1.jpg', 'rb')
f2 = open ('2.jpg', 'wb')
bytes = f1.read()
f2.write (bytes)
f1.close()
f2.close()
```

Random accessing of binary files using mmap :—

mmap :- memory mapped file is a module in python that is useful to map/link to a binary file and manipulate the data of file as we do with the strings.

→ Means once binary file is created with some data, that data is viewed as strings and can be manipulated using mmap module.

Syntax : `mm = mmap (f.fileno(), 0)`

→ This will maps currently open file f with object 'mm'.

vision

@python.hub

→ First argument of mmap() method is fileno is a handle to file object. f represents actual binary file that is being mapped.

→ The second argument is zero represents total size of file should be considered for mapping, if size=0 then it will consider whole life else specefied number of bytes.

→ We can read data from file using methods like:

```
print (mm.read())
print (mm.readline())
```

→ Also possible to retrieve content of file using slicing operator as:

```
print (mm[5:])      # display from 5th byte to end
print (mm[5:10])    # display 5th to 9th byte
```

→ Replace content using slicing

```
mm[5:10] = str
```

→ To search a string:

```
n = mm.find ('hello')    # return position of hello in file
```

Note:
Memory mapping is done only for binary files and not for txt files

→ To convert strings to binary (bytes) format before they stored to a file. It can be done as...

```
name = name.encode()    # convert name string to bytes
```

@python.hub

→ So similarly while reading a we get binary string from file.

The binary string can be converted to ordinary string using....

```
dob = dob.decode()
```

Ex: Program to convert student.txt — binary file to store names and date of birth (dob).

```
n = int (input ("How many records:"))
with open ('student.txt', 'wb') as f1:
    for i in range (1, n+1):
        name = input ('Enter name:')
        dob = input ("Enter dob:")
        name = name.encode()
        dob = dob.encode()
        f1.write(name+dob)
```

O/p:

```
How many records: 2
Enter name: Arjun
Enter dob : 01-01-1993
Enter name: Vivek
Enter dob : 02-02-1993
```

student.txt

```
Arjun01-01-1993Vivek02-02-1993
```

@python.hub

Ex: Python program to retreive name and dob (mm-dd-yyyy) of students from given file student.txt.

```
import mmap
import sys
print ("1. to display all entries")
print ("2. to display dob of specefic student")
print ("3. to modify dob")
print ("4. to exit")
ch = int (input ('Enter you choice'))
if (ch == 4):
    sys.exit()
with open ('student.txt', 'r+b') as f1:
    mm = mmap.mmap (f1.fileno(), 0)
    if (ch == 1):
        print (mm.read().encode())
    if (ch == 2):
        name = input ("Enter name:")
        n = mm.find (name.encode())
        n1 = n + len(name)
        dob = mm[n1 : n1+10]
        print ('dob:', dob.decode())
    if (ch == 3):
        name = input ("Enter name:")
        n = mm.find (name.encode())
        n1 = n + len (name)
        dob1 = input ('Enter new dob:')
        mm [n1 : n1+10] = dob1.encode()
mm.close()
```

VISION

@python.hub

o/p 1:

1. to display all entries
2. to display dob of specific student
3. to modify job
4. to exit

Enter your choice: 1

Arjun 01-01-1993 Vivek 02-02-1993

o/p 2:

1. to display all entries
2. to display dob of specific student
3. to modify job
4. to exit

Enter your choice: Vivek

dob: 02-02-1993

o/p 3:

1. to display all entries
2. to display dob of specific student
3. to modify job
4. to exit

Enter your choice: 3 Arjun.

Enter name: Arjun

Enter new dob: 24-09-1993

vision

@python.hub

CLASS AND OBJECT ORIENTED PROGRAMMING

CLASS

→ A class is a way to bind data and it's associated function together.

→ Entire set of data and code of an object can made a user-defined data type. with the help of class.

→ Objects are the variables of type class.

→ Once a class has been defined, we can create any number of objects belonging to that class.

→ Thus class is a collection of objects of similar type.

→ A class can be created using 'class keyword'. A class describes attributes and actions performed by it's objects.

```
Syntax:    class    class-name:
               "'optional class documentation string"'
               class-suite  # variables and function definiti
```

→ A class creates a local namespace where attributes are defined. Attributes may be data or functions, functions which are member of class is known as member functions.

→ The class suite consists of variables and functions which are known as data members and member function.

VISION

@python.hub

ATTRIBUTES

→ Attributes are known as data members. There a two type of attributes :
1. class attributes.
2. Instance attributes.

→ class attributes are attributes which are created by class itself, whereas instance attributes are created by particular instance of class (object)

Ex : Shows creation of class attributes and instance attributes.

```
class student :
        s_name = "Arjun"   # class attribute
s1 = student()
s1.r_no = 56    # instance attribute
p2 = product()
p2.r_no = 79    # instance attribute
print('Product name :',
print('Student name :', s1.s_name)
print ('Student r_no :', s1.r_no)
print ('Student name :', s1.name)
print ('Student r_no :', s2.r_no)
```

o/p : Student name : Arjun
Student r_no : 56
Student name : Arjun
Student r_no : 79

vision

@python.hub

→ class attributes can be accessed by any object of the class. But class or object

→ $2 cannot access instance attributes created by $1. If you want to access attribute created by $1 using $2, it will throw an error.

Ex: $1. s1.contact = 9825

Now contact attribute is instance attribute of s1 and only s1 can access it. s2 cannot access it.

print(s2.contact) # throws an error.

Note: Variable declared in class are called class variable / static variable and variable declared in the constructor are constructer variable

CLASS METHODS

→ There are 4 types of class methods:

1. Constructor:
2. Member function
3. Destructor
4. Class Methods
5. Static Methods.

VISION

@python.hub

1. Constructor:

In python, __init__() method acts as a constructor, this being called as soon as the object of class is created. In python name of constructor is fixed.

→ The __init__() method is used to initialize the instance of class. There is no explicit constructor method.

→ Method is also known as member function. A method is just a function which is defined inside the class.

Ex:

```
class product:
    def __init__(self, name, price):
        self.name = name
        self.price = price
    def display(self):
        print(self.name)
        print(self.price)
p1 = product('Computer', 20000)
p2 = product('Table', 2000)
p1.display()
p2.display()
```

O/p:
Computer
20000
Table
2000

→ Here self is a reference to the instance/object being constructed:

Types:

1. default Constructor: It doesn't accept any argum It's definition has only one argument which is reference to instance being created (self).

VISION

@python.hub

Ex: def __init__(self):

2. Parameterized constructor:

The parameterized constructor takes its first argument as a reference to the instance being constructed known as self and the rest of the arguments are provided by programmer

Ex: def __init__(self, name, age):

2. Member Function:

→ Method is also known as member function which is defined inside a class.

→ Method of class should have first parameter 'self' which is used as a reference to the instance.

→ In python 'self' is not a keyword, it is a naming convention, hence 'self' is a variable. it's name could be any but should follow rules of identifiers.

```
Ex:
class product:
    count = 1
    def display(self):
        print(product.count)  # class attribute
p1.product()
p1.displaycount()
```

@python.hub

3 Destructors :-

→ Similar to constructor there is no explicit destructor.
→ The --del--() method acts as a destructor.
→ destructor method is called when the object don't have any other references to it and when it is about to destroyed.

Ex: Demonstration of destructor

```
class product:
    count=1
    def __init__(s, name, price):
        s.name=name
        s.price=price
        s.count=product.count
        product.count += 1
    def display(s):
        print(s.name)
        print(s.price)
        print(s.count)
    def __del__(self):
        print('product destroyed')
p1=product('Computer', 20000)
p2=product('Table', 200)
p1.display()
p2.display()
print('deleting p1')
del p1
print('deleting p2')
del p2
```

O/P:

```
Computer
20000
1
Table
200
2
deleting p1
Product destroyed
deleting p2
product destroyed
```

VISION

@python.hub

4 Class Method:

→ These methods act on class level. class methods are methods which act on the class variable or static variables.
→ These method are written using @classmethod decorator.
→ By default. first parameter of class method is 'cls'. which refers to the class itself,
For eg, 'cls.var' is the format to refer to the class variable.
→ These method are generally called. using classname.method()
→ The processing which is commonly needed by all instances of class is handled by class methods

Ex: Demonstration of class method.

```
class Bird:
    wings = 2
    @classmethod
    def fly(cls, name):
        print('{} flies with {} wings'.format(name, cls.wings))

Bird.fly('sparrow')
Bird.fly('Pigeon')
```

O/p:

```
Sparrow flies with 2 wings
Pigeon flies with 2 wings.
```

@python.hub

5 Static Method :

→ Static methods are needed when processing it at class level but we did not involve the class or instances.

→ static methods are useful when some processing is related to the class but does not need the class or its instances to perform any work.
For example, setting environment variables, counting no. instances of class or changing attributes of another class.

→ Also static methods are useful when some processing is related to the class but does not need the class or its instances to perform any work.
For example, setting environment variables, counting number of instances of class or changing the attributes of

→ Also static methods can be used to accept some values, process them and returns result. In this case the involvement of neither class not object is needed

→ static methods are written using decorator @staticmethod.

→ And static methods, can be called using classname.staticmethod_name()

ision

@python.hub

Ex: Demonstration of static method:

```
class myclass:
    n=0
    def __init__(self):
        myclass.n = myclass.n+1
    @staticmethod
    def noof_objects():
        print('no. of instances created:', myclass.n)

obj1 = myclass()
obj2 = myclass()
obj3 = myclass()
obj4 = myclass()
myclass.noof_objects()
```

o/p:

```
no. of instances created: 4
```

@python.hub

Ex:. Class Example.

```
class student:
    Studentcount = 0          # 0 in zero
    def __init__(self, rollno, name, course):
        self.rollno = rollno
        self.name = name
        self.course = course
        student.studentcount += 1
    def displaycount(self):
        print("Total students =", student.studentcount)
    def displaystudent(self):
        print('Roll no :', self.rollno)
        print('name :', self.name)
        print('course :', self.course)
st1 = student(1, 'Arjun', 'EC')
st2 = student(2, 'Vivek', 'EE')
st1.displaystudent()
st2.displaystudent()
st2.displaycount()
```

O/p :

```
Roll no : 1
name : Arjun
course : EC
Roll no : 2
name : Vivek
course : EE
Total students = 2
```

vision

@python.hub

→ In this program, studentcounter is a class variable which is shared among all instaces of class. It can be accessed from inside / outside class by giving student.studentcount.

→ The first method __init__() is a special method, which is called class constructor or initialization method that python ~~class~~ calls when we create new instance / object of this class.

→ Class method have one difference from ordinary functions - they must have an extra argument in begining of parameter list. This particular argument is used for referencing instance / object.

→ If we have method with no arguments, then we still have to define the method that have self argument. self is instance identifier and is required so that the statements within the method can have automatic access to current instance attributes.

CREATING OBJECTS

→ An object is created by calling the class name with arguments defined in __init__() method. We can access methods by using . (dot) operator along with object.

Ex:
```
st1 = student (1,'kevin','comp')
st2 = student (2,'Arjun','EC')
```

@python.hub

→ If class does not contains __init__() function then object can be generated using following syntax:

Object_name = class name()

BUILT-IN ATTRIBUTE METHODS

→ The are built in functions to access attributes. Using these functions we can access attribute values, check whether attribute exists or not, modify an attribute value or delete an attribute.

→ The following gives built-in attribute methods:

1. getattr(obj, name[, default]):
This method is used to access the attributes of object obj.

2. hasattr(obj, name):
This method is used to check if an attribute exists or not.

3. setattr(obj, name, value):
This method is used to set an attribute. If attribute does not exists then it will be created.

4. delattr(obj, name):
This method is used to delete an attribute.

ision

@python.hub

Ex: Program for use of attribute method.

```
class student:
    def __init__(self,r_no,n,cou):
        self.r_no = r_no
        self.n = n
        self.cou = cou
    def display(self):
        print('Roll no:', self.r_no)
        print('name:', self.n)
        print('course:', self.cou)
st1 = student(1,'Arjun','EC')
st1.displaystudent()
at = getattr(st1,'name')
print("getattr(st1,'name')", at)
print("hasattr(st1,'age')", hasattr(st1,'age'))
print("setattr(st1,'age',21):")
setattr(st1,'age',21)
st1.displaystudent()
print('age:', st1.age)
st1.weight = 62
print('weight:', st1.weight)
print('delete attribute weight')
delattr(st1,'weight')
print(st1.weight)
```

o/p:

```
Roll no: 1
name: Arjun
course: EC
getattr(st1,'name'): Arjun
hasattr(st1,'age'): false
setattr(st1,'age',21):
Roll no: 2
name: Arjun
course: EC
age: 21
weight: 62
delete attribute weight
NameError: - - - - -
```

as it is already deleted, we cannot print it.

@python.hub

NAMESPACE

→ Namespace refers a block of memory where names are mapped (or linked) to objects.

→ Suppose we write n = 10 - here 'n' is name given to integer object 10.

→ A class maintains its own namespace, called class namespace. In the class namespace the names are mapped to class variables, similarly every instance of class have it's own namespace called 'instance namespace'.

→ In instance namespace the names are mapped to the instance variable.

→ If class variable can be modified in class namespace then changes will reflected to all instances of class.

→ If class variable is modified in specific instances namespace then change will reflect to the specific instance only.

Ex: Demonstration of modifying class variable in class namespace.

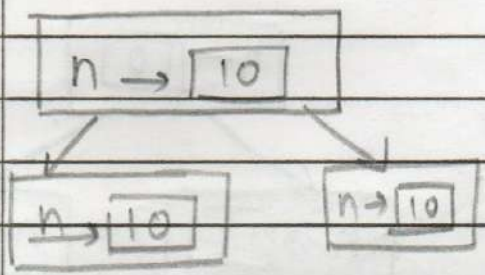
```
class student:
    n=10
print(student.n)
student.n+=1
print(student.n)
s1.student()  # modifying class variable for all instances
print(s1.n)
s2=student()
print(s2.n)
```

O/P

```
10
11
11
11.
```

@python.hub

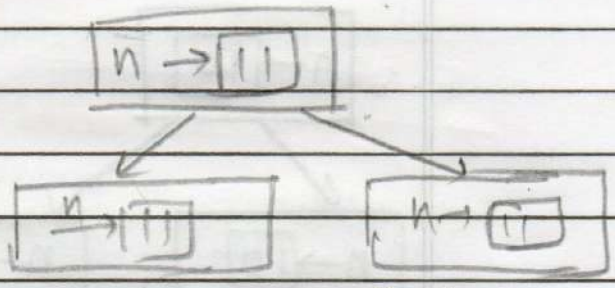
before modifying class variable n



instance namespace of s1.

instance namespace of s2.

After modifying class variable n



instance namespace of s1

instance namespace of s2

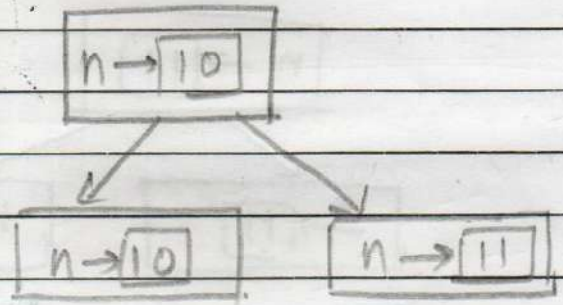
Ex: demonstration of modifying class variable in instance namespace.

```
class student:
    n=10
# modifying class variables is seen in s1 instance
s1=student()
print(s1.n)
s1.n+=1
print(s1.n)
# modified class variables not seen in another instance
s2=student()
print(s2.n)
```

o/p:
10
11
10

@python.hub

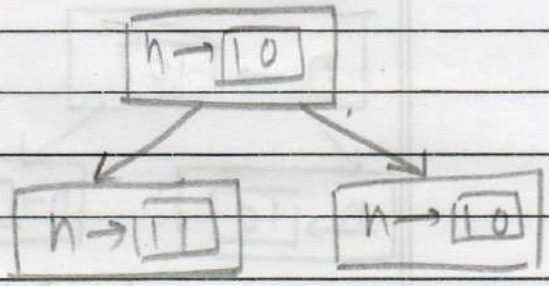
Before modifying class variable n



Instance namspace of S1

Instance namspace of S2

After modifying class variable n



Instance namespace of S1

Instance namspace of S2.

BUILT IN CLASS ATTRIBUTE

Python contains several built in class attributes. These attributes are accessed using . (dot) operator.

1. __dict__ : This attribute contains dictionary containing the class's namespace
2. __doc__ : It describes the class or it contains the class documentation sting
3. __name__ : This attribute contains class name.
4. __module__ : This contains the module name in which the class is defined
5. __bases__ : This contains an empty tuple containing base classes, in order of their occurences in the base class list.

vision

@python.hub

Ex: Program for use of class attribute

```
class student:
    'common base class for all students'
    def __init__(self, rollno, name, course):
        self.rollno = rollno
        self.name = name
        self.course = course
    def displaystudent(self):
        print('Roll no:', self.rollno)
        print('name:', self.name)
        print('course:', self.course)
st1 = student(1, 'Arjun', 'comp')
st1.displaystudent()
print(student.__doc__)
print(student.__name__)
print(student.__module__)
print(student.__bases__)
print(student.__dict__)
```

o/p: Roll no: 1
name: Arjun
course: Comp
common base class for all students.
student.
__main__
(<class 'object'>,)
{'__module__': '__main__', '__doc__': 'common base for all students', '__init__': <function student...
}

VISION

@python.hub

ACCESS SPECIFIER

→ To access data member of class python provides following access specifiers.
1. Public
2. Private
3. Protected

→ Public attributes can be accessed anywhere in the program. Protected attributes can be accessed by implementing inheritance.
In which protected data member of base class can be accessed by the child class object.

→ Private data members can be accessed inside the particular class only.

→ Private provides special naming conventions for defining such attributes, by default variables are declared as public.

Naming	Type	Description
var_name	public	Attributes can be used anywhere
_varname	protected	Variables cannot be used outside the class, unless it is inside subclass.
__varname	Private	These are accessible only inside the class.

→ Adding '_' as prefix, makes it protected.
→ Adding '__' as prefix, makes it private.

vision

@python.hub

Ex: Program for use of access specifier.

```
class test:
    def __init__(self):
        self.__priv = "Private member"
        self._prot = "Protected member"
        self.pub = "Public member"
    def display(self):
        print('inside class')
        print(self.__priv)
        print(self._prot)
        print(self.pub)
t = test()
t.display()
print('outside class')
print(t.pub)
print(t._prot)
print(t.__priv)
```

o/p: Inside class
Private member
Protected member
Public member
Outside class
Public member
AttributeError:

@python.hub

OOP CONCEPTS (Class And object already covered)

1. ENCAPSULATION

→ It is most basic concept of OOP. It is combining data and functions associated with data in single unit.
In most of languages including python - this unit is known as class.

→ Thus we can say that encapsulation is implemented using class.

→ Even data members of class can also be accessed using member functions only (In case of private and protected data members)

- It keeps data safe from external interference and misuse.

→ We have already seen many examples on class

2. DATA HIDING

→ We can hide data by using private variable in class

→ Data hiding can be defined as the mechanism of hiding the data of a class from the outside world or to be precise from other classes.

→ This is done to protect data from accidental modification or intentional access.

vision

@python.hub

Ex:

```
class hidedemo:
    __num = 0
    def numbercount(self):
        self.num+ = 1
        print('Number count=', self.__num)
n = hidedemo()
n.numbercount()
print(n.__num)  #leads to error due to private.
```

O/p:

```
number_count = 1
Traceback (most recent call last):
  ...
AttributeError: ...
```

3. INHERITANCE

→ The mechanism of providing new class from old class is known as inheritance.

→ The old class is known as base class or parent class or super class

→ The "new one is called child class or derived class or sub class.

→ Inheritance allows subclass to inherit all variables and methods of parent class.
(in case of public & protected, however private data is not accessible in derived class).

@python.hub

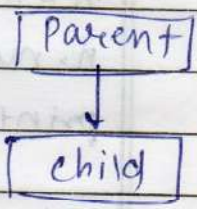
Types of inheritance :

A : SINGLE INHERITANCE.
B : MULTILEVEL INHERITACE
C : MULTIPLE INHERITANCE.

A : SINGLE INHERITANCE

Syntax :

```
class subclass_name (parent_class) :
        class suite.
```



Ex :

```
class parent :
        v1 = "Value 1"
        v2 = "Value 2"
class child :
        pass.
p = parent()
print ('Using parent object')
print(p.v1, p.v2)
c = child()
print ('Using child object')
print (c.v1, c.v2)
```

o/p :

```
Using parent object
Value 1 Value 2
Using child object
Value1 Value 2.
```

vision

@python.hub

Constructor in inheritance

Program to create two classes father and son where father class having constructor where a variable property is initialized with 80000, also inherit son from father.

Code:

```
class father:
    def __init__(self):
        self.property = 80000
    def display(self):
        print("father property=", self.property)
class son(father):
    pass.
s = son()
s.display()
```

o/p:

```
Father property = 80000
```

Note: Like variables and methods, constructor of super class also available to sub class object by default.

@python.hub

Overriding subclass constructors and methods

→ When programmer write constructor to ~~subjects~~ subclass, the superclass constructor is not available to sub class. In this case only sub class constructor is accessible using sub class object.

→ That means sub class constructor is replacing the super class constructor which is known as constructor overriding.

→ Similarly in subclass if we write a method which name is same as that superclass method, then it will overwrite the superclass's method.

Ex:

```
class father:
    def __init__(self):
        self.property = 80000
    def display(self):
        print("Father property =", self.property)
class son(father):
    def __init__(self):
        self.property = 20000
    def display(self):
        print('son's Property =", self.property)
```

```
s = son()
s.display()
```

o/p:

Son's Property = 20000

vision

@python.hub

IMP

ACCESSING SUPER CLASS CONSTRUCTOR:
The super() method

→ Any constructor written in super class is not available in sub class if sub class has a constructor. Then how can we initialize the super class instance variable. and use them in sub class?

→ super() is inbuilt method which is useful to call the super class constructor or method from sub class.

→ Super class constructor is called using super() method from inside sub class constructor.
It can be used as:

```
super.__init__(arg) # call superclass constructor.
super.method()      # call superclass method.
```

→ When there is a constructor in super class ~~method~~ with parameter then we have to create another constructor with parameter in sub class & call super class constructor using super() from sub class constructor.

```
# sub class constructor:

def __init__(self, p1=0, p2=0)
    super.__init__(p2)  # send p2 value to superclass Constructo
    self.p1 = p1
s = son(2, 8)
```

VISION

@python.hub

Ex: Overriding subclass constructor and method using super().

```
class father:
    def __init__(self, p2=0)
        self.p = p2
    def display(self):
        print("father's property =", self.p)

class son(father):
    def __init__(self, p1=0, p2=0)
        super().__init__(p2)
        self.p1 = p1
    def display(self):
        super().display()
        print("child's property =", self.p1)

s = son(80000, 20000)
s.display()
```

o/p:

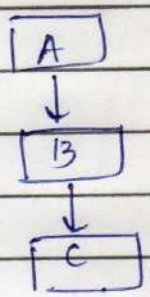
```
father's property = 20000
child's property = 80000
```

vision

@python.hub

B: MULTILEVEL INHERITANCE

→ A mechanism of ~~derived~~ deriving a class from another derived class is called multilevel inheritance



→ class B derived from class A
→ class C derived from classB.

Ex:

```
class student:
    def getno(self,no):
        self.no=no
    def putno(self):
        print('R_no:', self.no)
class test(student):
    def getmarks(self,sub1,sub2)
        self.sub1= sub1
        self.sub2= sub2.
    def putmarks(self):
        print("sub1 marks:",self.sub1)
        print("sub2 marks:",self.sub2)
class result(test):
    total=0
    def display(self):
        result.total = self.sub1 +self.sub2.
        self.putno()
        self.putmarks()
        print("Total marks:", result.total)
```

@python.hub

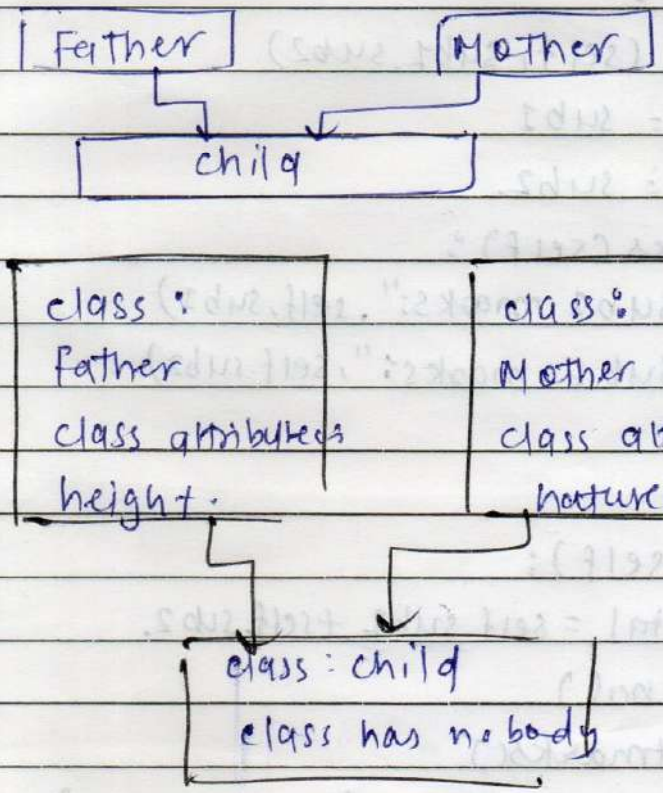
```
r = result()
r.getno(2)
r.getmarks(60,50)
r.display()
```

```
o/p:
R_no : 2
Sub 1 marks : 60
Sub2 marks : 50
Total marks : 110
```

C: MULTIPLE INHERITANCE

Syntax:

```
class subclass name (Parent_class 1, Parent_class 2.):
    class suite.
```



vision

@python.hub

Ex:

```
class father:
    def height(self):
        print('Height:6')
class mother:
    def nature(self):
        print('Nature:kind')
class child(father, mother):
    pass
c = child()
c.height()
c.nature()
```

o/p:

```
Height:6
Nature:kind
```

CONSTRUCTOR IN MULTIPLE INHERITANCE : PROBLEM

```
class A:
    def __init__(self):
        self.a='a'
        print(self.a)
class B:
    def __init__(self):
        self.b='b'
        print(self.b)
class C(A,B):
    def __init__(self):
        self.c='c'
        print(self.c)
        super().__init__()

ob = C()
```

o/p:

```
c
a
```

@python.hub

Limitation:

only left parent's constructor will call using super()

Note: here class C calls constructor of only A class, not B. If we write class C as class C(B,A): then it will call constructor of only B class and output will be c b, but it will not call constructor of both base class.

SOLUTION:

```
class A():
    def __init__(self):
        self.a = 'a'
        print(self.a)
        super().__init__()
class B():
    def __init__(self):
        self.b = 'b'
        print(self.b)
        #super().__init__()
class C(A,B):
    def __init__(self):
        self.c = 'c'
        print(self.c)
        super().__init__()
ob = C()
```

O/P:

```
c
a
b
```

VISION

@python.hub

Note : if class which calls super() method have no parent class then it will call constructor of itself. But that constructor will call only once due to MRO and after that it will call constructor of right Base class.

IMP Method Resolution Order (MRO)

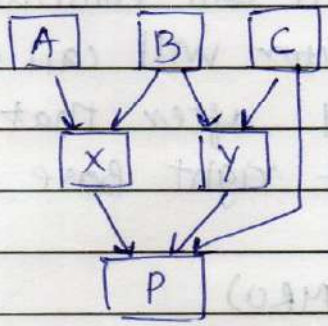
In multiple inheritance, any specified attribute or method is scanned first in the current class.
If not found, the search continues in parent classes in Depth first manner, from left to right without searching same class again.
Searching in this way is called method of resolution

Three principles followed by MRO :

1. To search for sub class before going for its base class. Thus if B is inherited from A, It will search B first and then goes to A.
2. When class is inherited from several classes, it searches it in order from left to right in the base classes. for example C is inherited as class C(A,B) the after searching in C, A will be searched and if attribute not found then searching in B is done
3. It will not visit any class more than once. That means traversal will only perform once.

@python.hub

consider case given below:



MRO gives clear idea regarding which classes are executed in which sequence.

To know the MRO, we can use mro() method as:

classname.mro()

: It returns sequence of execution of classes:

Ex:

```
class A:
    def method(self):
        print('A class')
        super().method()
class B:
    def method(self):
        print('B class')
        super().method()
class C:
    def method(self):
        print('C class')
class X(A,B):
    def method(self):
        print('X class')
        super().method()
class Y(B,C):
    def method(self):
        print('Y class')
        super().method()
class P(X,Y,C):
    def method(self):
        print('P class')
        super().method()

pob = P()
pob.method()
print(P.mro())
```

vision

@python.hub

O/p :

P class
X class
A class
Y class
B class
C class
[<class '__main__.P'>, <class '__main__.X'>, <class '__main__.A'>
, <class '__main__.Y'>, <class '__main__.B'>,
<class '__main__.C'>]

→ Here first P will be execute, after P, left parent of P, x will execute, after that x's left parent A will execute. since A' don't have parent, flow will be redirected to P. Then right parent P's next unvisited parent, Y will be executed, Y's left parent, B, so B will be executed.
Now B does not have parent, so flow will again redirected to P's next parent which is C, so C will be executed.
Now after execution of C, there is not any remaining unvisited parent of P, so traversal will be halt.

vision

@python.hub

Polymorphism

→ Polymorphism : ability to take more than one form.
→ An operation / function may exhibit different behaviours in different instances. the behaviours depends on the type of data used in operation.
→ for eg, consider operation of addition (+).
for two numbers it will generate addition.
for two strings it will generate concatenated string.

Examples of polymorphism in Python :

1. Duck typing philosophy of python.
2. Operator Overloading
3. Method overloading
4. Method overriding.

1. DUCK TYPING PHILOSOPHY

→ Python's type system is strong because every variable/object has a type that we can check with type() function.
→ Python's type system is dynamic since the type of variable is not explicitly declared but it changes with the content being stored
→ similarly if we want to call a method on an obje we do not need to check the type of object and we do not need check whether that method really belong to That object or not.

@python.hub

```
Ex: def call_talk(obj):
        obj.talk()
```

The call_talk method accepts object as an argument and according to the object passed, talk method is invoked

→ If object passed to this method belongs to 'Duck' class then talk() method of 'Duck' class will be called. If the object passed is of 'Human' class then talk() method of 'Human' class will be called.

Ex:

```
class Duck:
    def talk(self):
        print("Quack!")
class Human:
    def talk(self):
        print('Hello! Hi!")
def call_talk(obj):
    obj.talk()
x = Duck()
call_talk(x)
x = Human()
call_talk(x)
x = dog()
call_talk(x)
```

o/p:

```
Quack!
Hello! Hi!
NameError: name 'dog'
        is not defined.
```

vision

@python.hub

→ In python, we never worry about the type (class) of object. The object type is distinguished only at run time.

→ If it walks like Duck and talk like Duck then it must be 'duck' - This principle is known as 'duck typing'.
In the above example, behaviour of talk changes depending on object type.

Ex
```
class Duck:
    def talk(self):
        print('Quack!')
class Human:
    def talk(self):
        print('Hello! Hi!')
class Dog:
    def bark(self):
        print('Bow, Bow!')
def call_talk(obj):
    if (hasattr(obj, 'talk')):
        obj.talk()
    elif (hasattr(obj, 'bark')):
        obj.bark()
    else:
        print('wrong object')
x = Duck()
call_talk(x)
x = Human()
call_talk(x)
x = Dog()
call_talk(x)
```

O/p:
Quack!
Hello! Hi!
Bow, Bow!

vision

@python.hub

2. Operator Overloading :

→ Python allows to add two variables of user defined type (i.e., class) with the same syntax that is applied to basic types.

→ This means that python has ability to provide the operators with a special meaning for a data type. The mechanism of giving such special meaning to an operator is known as operator overloading.

→ Every operator in python internally written as a special method.

→ For `'+'` operator there is one internal method named - `__add__()` so if we add two numbers like `'a+b'`, the internal method is called

`a.__add__(b)`

→ By overriding this method to act upon objects, we can make '+' operator to succesfully act upon the objects also.

Ex :

```
a,b = 2,3
print(a+b)
#call
    (2.__add__(3)) internally.
```

@python.hub

→ Since we need to override, we have to write the method with same name but with object as

```
def __add__(self, other):
    return self.a + other.b
```

Here self.a represent numeric content of first class and other.b represent numeric content of other class.

→ The internal methods are known as magic method Similarly as '+' we can overload other operator by it's corresponding magic method.

Ex: Program to add class objects using overloading '+' operator.

```
class book1:
    def __init__(self, pages):
        self.pages = pages
    def __add__(self, other):
        return self.pages + other.pages

class book2:
    def __init__(self, pages):
        self.pages = pages

b1 = book1(100)
b2 = book2(100)
print('Total pages:', b1 + b2)
```

o/p.

Total pages: 200.

@python.hub

Note : It is not compulsory to make two classes we can also perform operation on two objects of same class using operator overloading.

Operators and their corresponding magic method.

Operator	Magic method
+	object.__add__(self, other)
-	object.__sub__(self, other)
*	object.__mul__(self, other)
/	object.__truediv__(self, other)
//	object.__floordiv__(self, other)
%=	object.__imul__(self, other)
/=	object.__idiv__(self, other)
//=	object.__ifloordiv__(self, other)
**=	object.__ipow__(self, other)
%=	object.__imod__(self, other)
<	object.__lt__(self, other)
<=	object.__le__(self, other)
>	object.__gt__(self, other)
>=	object.__ge__(self, other)

@python.hub

Ex: Python program to overload "==" operater to print the details of student having same marks.

```
class student:
    def __init__(self, name, marks)
        self.name = name
        self.marks = marks.
    def __eq__(self, other):
        return self.t_marks == other.t_marks
s1 = Student("Arjun", 100)
s2 = Student("Vivek", 200)
s3 = Student("Ramesh", 100)
if (s1 == s2):
    print("student 1 & student 2 have same marks")
elif (s1 == s3):
    print("Student 1 & Student 3 have same marks")
elif (s2 == s3):
    print("student 2 & student 3 have same marks")
else:
    print("Not same")
```

o/p:

Student 1 & Student 3 have same marks

vision

@python.hub

3. METHOD OVERLOADING:

If method is written in such that it can perform more than one task, it is called method overloading.

```
class myclass:
    def sum(self, a=None, b=None, c=None):
        if(a!=None and b!=None and c!=None):
            print('sum of three :', a+b+c)
        elif(a!=None and b!=None):
            print('sum of two:', a+b)
        else:
            print('Enter atleast two argument')
m= myclass()
m.sum(10,20,30)
m.sum(10,20)
m.sum(10)
```

o/p:

```
sum of three : 60
Sum of two : 30
Enter atleast two arguments.
```

vision

@python.hub

4 METHOD OVERRIDING:

→ When there is a method in super class, writing same method with same name in sub class. So That it replaces the superclass method is called method overriding

Ex:

```
import math
class square:
    def area(self,x):
        print('area of square :',x*x)
class circle(square):
    def area(self,x):
        print('Area of circle:',math.pi*x*x)
c= circle()
c.area(10)
```

o/p:

Area of circle : 314.1592653589793

→ Here area() method is overridden as area() method of square class (base class) is redefined in circle() class.

VISION

@python.hub

NEED OF ABSTRACT CLASS.

→ Consider program,

```
class myclass:
    def calculate(self, x):
        print('square :', x*x)
obj1 = myclass()
obj1.calculate(2)
obj2 = myclass()
obj2.calculate(3)
obj3 = myclass()
obj3.calculate(4)
```

O/P
square : 4
square : 9
square : 16

→ Now, in this program if we change behaviour method based on object like first object want to calculate square, second object want to calculate square root and third object want to calculate cube value.

ABSTRACT METHOD AND ABSTRACT CLASS.

→ An abstract method is a method whose action is redefined in sub class as per the requirement of the objects.

→ Generally abstract methods are written without body since their body will be redefined in the sub class anyhow. (It is also possible to write abstract method with body).

→ To make method abstract we should use decorator @abstractmethod.

→ On the other hand concrete method is a method with body.

@python.hub

→ Abstract class is a class that generally contains some abstract methods.
Since abstract class contains abstract methods whose implementation (body) is later defined on sub classes, it is not possible to estimate total memory required to create the object for the abstract class.
So PVM can not create objects to an abstract class.

Abstract method and Abstract class :

The way to create abstract class is to derive it from a meta class ABC that belongs to abc (abstract base class) module as :

```
from abc import ABC, abstractmethod. or
from abc import *
class Abstractclass_name(ABC):
```

Ex: To find square, squareroot, and cube using abstract method.

```
from abc import *
class myclass(ABC):
    @abstractmethod
    def calculate(self,x):
        pass
class sub1(myclass):
    def calculate(self,x):
        print('square:',x*x)
class sub2(myclass):
    def calculate(self,x):
        print('square root',sqrt(x))
```

vision

@python.hub

```
class sub3(myclass):
    def calculate(self,x):
        print('cube value:', x*x*3)

obj1 = sub1()
obj1.calculate(16)
obj2 = sub2()
obj2.calculate(16)
obj3 = sub3()
obj3.calculate(16)
```

OUTPUT:

square: 256
square root: 4.0
cube value: 4096

INTERFACES IN PYTHON

→ Abstract class is a class which contains some abstract methods as well as concrete method also.

→ Interface is an abstract class which contains only abstract methods.

→ None of method in interface have body.

→ Difference between abstract class and interface:
generally abstract class is written when some common features shared by all objects.
For example, take a class wholesaler which represent a whole sale shop. with textbooks and stationary like pens, papers and notebooks.

@python.hub

```
class Wholesale(ABC):
    @abstractmethod
    def text_books(self):
        pass
    @abstractmethod
    def stationary(self):
        pass.
```

→ Let's take retailer 1, a class which represents retail shop.
Retailer1 wants textbooks of X class and some pens.
Similarly Retailer 2 wants textbook of X class and some papers.

→ Here in both subclasses textbook need is same so on place of abstractmethod we should write text_books method as concrete method. so in this case. class Wholesaler becomes abstract class.

→ Now let's take another case:
If Retailer 1 wants books of X class and Retailer2 wants text books of IV class then book requirement of text books are different.
In this case text_books() method of wholesales becomes abstract method,
so due to all the methods are abstract, class becomes interface.

vision

@python.hub

Wholesaler class an abstract class :

```
from abc import *
class Wholesaler (ABC):
    def text_books (self):
        print ('order for text books of class X')
    @abstractmethod
    def stationary (self):
        pass
class Retailer1 (Wholesaler):
    def stationery (self):
        print ('order for pens')
class Retailer2 (Wholesaler):
    def stationery (self):
        print ('order for papers')

r1 = Retailer1()
r2 = Retailer2()
print ('Retailer 1')
r1. text_books()
r1. stationery()
print ('Retailer2')
r2. text_books()
r2. stationery
```

o/p :

```
Retailer1
Order for text books of class X
Order for pens
Retailer2
Order for text books of class X
Order for papers
```

@python.hub

Wholesaler class as an interface

```
from abc import *
class Wholesaler(ABC):
    @abstractmethod
    def text_books(self):
        pass
    @abstractmethod
    def stationery(self):
        pass
class Retailer1(Wholesaler):
    def text_books(self):
        print('order for text books of class X')
    def stationery(self):
        print('order for pens')

class Retailer2(Wholesaler):
    def text_books(self):
        print('order for text books of class IV')
    def stationery(self):
        print('oreder for papers')
r1=Retailer1()
r2=Retailer2()
print('Retailer 1')
r1.stationery()
r1.text_books()
print('Retailer 2')
r2.text_books()
r2.stationery()
```

O/P.

```
Retailer 1
order for pens
Oreder for text books of class X
Retailer 2
Order for text books of class IV
Order for papers.
```

vision

@python.hub

MODULES

→ Module is a file containing python statements and definition.
→ A module is a python object with arbitary named attributes and can bind and reference.
→ We can use module in python to break down large program in to small manageable and organized files.
→ A module can define functions classes and variable. A module can also include runnable code.
→ Python has a lot of standard modules available.

Creating Modules:

→ We can define our most ~~de~~ used function in a module and import it
→ Any file containing a python code and with .py extension then it is a module.
→ For eg, there is a file 'prime.py' → It is called a module and module name python.

Import statement

→ We can use any python source code file as a module by executing an import statement
→ User defined modules can be imported the same way as we import built in modules.

Syntax: import module1 [, module 2, module 3, ...]

@python.hub

import with renaming

→ We can import built-in or user defined modules with an alias name.
The following code shows how built-in module math can be imported using alias name 'm'

```
import math as m
print(m.pi)
o/p: 3.141592653589793
```

→ This can save typing time.

from .... import statement

→ We can import specific names from a module without importing the module as a whole. The module math contains a lot of built-in functions. But if we want to import only the pi variable the from statement is used

```
from math import pi
print(pi)
o/p: 3.141592653589793
```

→ We can also import multiple attributes :-

```
from math import pi, sqrt
```

vision

@python.hub

Import all names

→ We can import all names from a module using following construct.

```
from math import *
```

PYTHON math MODULE

→ Python has a built in module that you can use for mathematical ~~conste~~ tasks.

Math methods.

ceil(x)	Returns a smallest integer greater than or equal to x. ceil(4.5) = 5
fabs(x)	Returns absolute value of x. fabs(-3) = 3.0
factorial(x)	Returns the factorial of x. fact(3) = 6
floor(x)	Returns largest integer less than or equal to x. floor(4.5) = 4
fmod(x,y)	Returns the remainder when x divided by y. fmod(14.5, 3) = 2.5
fsum(sequence)	Returns an accurate floating point sum of values in the iterable. fsum([1.5, 2.4, -3.3]) = 0.600000000000001
degrees(x)	Convert angle x from radian to degree. degrees(3.14159) = 179.958...
radians(x)	Convert angle x from degree to radian. radians(180) = 3.14159

@python.hub

sin(x).	Returns sine of x, x in radians
cos(x)	Returns cosine of x
tan(x)	Returns tangent of x
exp(x)	Returns e^{x} x
modf(x)	Returns fractional & integer part of x. mod (2.58) = (0.58, 2.0)
log10(x)	Returns base-10 logarithm of x
pow(x,y)	Returns x raised to power y
sqrt(x)	Returns squareroot of x.
trunc()	Returns truncated integer part of number
isnan(x)	Returns True if x is Not a number.

Constants on math module.

e	Returns Euler's number (2.7182...)
inf	Returns floating point positive infinity
nan	Returns floating point NaN
pi	Returns PI (3.1415.)
tau	Returns tau (6.2831...)

vision

@python.hub

OS Module.

→ The os module in python provides a way of using operating system dependent functionality.
The functions that the os module provides allows you to interface with the underlying operating system that python is running on - be that Windows, Mac or linux.

1. os.system('command name'):
For executing a shell command
Ex: To print directories commands is ls: so
os.system('ls')

2. os.environ():
To get the user's environment

3. os.getpid(): Returns the real process ID of current process

4. os.uname(): Returns information identifying the current operating system username.

5. os.listdir(path): Returns the list of entries on the directory given by the path.

6.

@python.hub

6 os.getcwd(): This method displays the current working directory.

Syntax:
```
import os
os.getcwd()
```

Ex:
```
import os
os.get
```

Ex:
```
import os
print(os.getcwd())
```

o/p:
E:\python

7 os.rename(): Renaming a file.

Syntax: os.rename(current_file_name, new_file_name)

Ex:
```
import os
os.rename('f.txt', 'newf.txt')
```

8 ~~Deleting~~ os.remove(): Used for deleting a file

Syntax: os.remove(file_name)

Ex:
```
import os
os.remove('hello.txt')
```

9 os.mkdir(): To create directory in the current directory.

Syntax: os.mkdir("directory_name)

Ex:
```
import os
os.mkdir("folder3")
```

vision

@python.hub

11. os.chdir() : To change the current working directory

Syntax: os.chdir(new-directory_name) or
os.chdir(path)

Example:

```
import os
os.chdir("Fruits")
```

12. os.rmdir() : Used for deleting a directory.

Syntax: os.rmdir(dirname)

Example:

```
import os
os.rmdir('Fruits')
```

13. os.rmdirs() : Recursively removes all directories of path mentioned

Syntax: os.rmdirs('mysub/mysub1/mysub3')

CALENDAR MODULE

Calendar module contains various functions related to calendar.

1. calendar.firstweekday() :
Returns the current setting for weekdays that starts each week. By default when calendar is first imported, first week-day is 0-Monday.

2. calendar.setfirstweekday(weekday) : Set the first day of each week to week day with the parameter passed. week day codes are (0) Monday to 6 (Sunday).

@python.hub

3. calendar.isleap(year): Returns true if a year is a leap year, False otherwise.

4 calendar.leapdays(y1,y2): Returns total number of leap days in the years within the range (y1,y2)

5 calendar.TextCalendar([firstweekday]):
This class can be used to generate plain text calendars.

6. calendar.TextCalendar(calendar.SUNDAY).formatyear(ye
Returns a m-column calendar for an entire year

7 calendar.TextCalendar(calendar.SUNDAY).formatmonth(year, month)
Returns a calendar with specified month and year.

8 calendar.calendar(year,w=2,l=1,c=6):
Returns a multiline string with a calendar for year formatted into three columns seperated by c spaces. w is a width in characters of each date and l is the number of lines for each week.

9. calendar.month(year,month): display calendar of specified year and month.

vision

@python.hub

REGULAR EXPRESSIONS (REGEX)

→ Regular expression is a string that contains special symbols and characters to find and extract information need by us from the given data.
A regular Expressions help us to search information, match, find and split information as per our requirements.

→ In python regular expression are also known as regex.

→ Python provides 're' module that stands for regular expression.
This module contains many methods which can be used to finding information in the available data.

reg = r'm\w\w'
Here prefix 'r' represents raw string of regular expression.
generally we write regular expressions as raw strings.

```
str = 'This is normal\n string
print (str)
```

o/p: This is normal
string

→ In normal string '\n' is interpreted as new line while in raw string '\n' is not interpreted

```
str = r'This is raw\n string'
print (str)
```

o/p: This is raw\n string

vision

@python.hub

Regular expression character and its Description

Character	Description
\d	Represents any digit (0-9)
\D	Represent any non-digit
\s	Represent white space Ex: \n, \t, \v
\S	Represent non-whiteSpace character
\w	Represents any alphanumeric (a-z, A-Z, 0-9)
\W	Represents non alphanumeric
\b	Represents space around words
\A	Matches only at start of string
\z	Matches only at end of string

Methods of re module used in Regular Expression

1. **match()** :
It searches a string in the beginning of input string and if match is found then returns object that contains resultant string otherwise it returns None.
We can access the string from returned object using group() method

2. **search()** :
This method searches string from beginning till the end and returns the first occurrence of the matching string. We can use group() method to retrieve the string from object returned by search()

@python.hub

3. findall():
Searches a string from beginning till end and returns all occurrences of matching string in the form of list object. If matching string are not found then it returns empty list.

4 split():
split the string according to the regular expression and resultant pieces are returned as a list.
If there are no string pieces, then it returns empty list.

5. sub():
substitutes / replaces new string in the place of existing strings. After substitution, the main string is returned by this method.

To create regular expression to search for strings starting with m and having total 3 characters using.

Code 1:
```
str = 'cat mat bat rat cat mat'
result = re.search(r'm\w\w', str)
if result != None
    print(result.group())
```

O/p:	search() returns only first matched occurrences instead of all occurrences.
mat	

VISION

@python.hub

Code 2:

```
import re
str = 'cat mat bat rat cat mat'
result = re.findall(r'm\w\w', str)
print(result)
```

O/p:

['mat', 'mat']

findall() returns list all occurrences of string matching with RE

Code 3:

```
import re
str1= 'cat mat bat rat cat mat'
str = 'mat bat rat cat mat'
result = re.match(r'm\w\w', str)
result1 = re.match(r'm\w\w', str1)
print(result)
if result != None:
    print(result.group)
print(result1)
```

O/p:

<re.Match object; span=(0,3), match='mat'>
mat
None.

match() returns only first matched occurrence instead of all occurrences.
It only returns match string if original string str begin with 'm' followed by two alphanumeric character else return NULL.

VISION

@python.hub

Python program to create regular expression to split a string into pieces where one or more non alphanumeric characters are found.

```
import re
str='This, is the; "core" python's book'
result=re.split(r'\W+',str)
print(result)
```

o/p:
['This','is','the','core','python','s','book']

Here \W indicates non alphanumeric character and + indicates one or more occurrences.

Python program to create regular expression to replace a string with a new string.

```
import re
str='Kumbhmela will be conducted at Ahmedabad
    in India'
result=re.sub(r'Ahmedabad','Allahabad',str)
print(result)
```

o/p: Kumbhmela will be conducted at Allahabad in India

sub() method has three arguments sub('re_original_string', 'replacement_string', 'input string from which searching will perform')
sub will replace argument 1 with argument 2 in str and return resultant string str.

@python.hub

Quantifiers in Regular Expressions.

Character	It's Description
+	1 or more occurrences of preceding regexp
*	0 or more occurrences of preceding regexp
{m}	Exactly m occurrences of preceding regexp
{m,n}	From m to n occurrences, m default 0, n infinity
\	Escape sequence special character
.	Matches with any character except newline.
^	Matches beginning of string
$	Matches ending of string / line
[...]	Denotes a set of possible character like [A-Z] matches with any single character from A to Z.
[^...]	Matches with every character except one inside brackets. Example [^A-Z] will match with any single digit or special symbol or lowercase alphabet.
(...)	Matches with regular expression inside parenthesis and the result can be capture (generally useful for grouping of r.e.'s)
R|S	Matches with regex either R or S

vision

@python.hub

Regular expression to retrieve all words starting with 'a' in a given string

```
import re
str = 'An apple a day keeps the doctor away'
result = re.findall(r'a[\w]*', str)
print(result)
```

o/p: ['apple', 'a', 'ay', 'away']

RE to retreive all words starting with a numeric digit.

```
import re
str = 'The meeting will be conducted on 1st and 21st
       of every month'
result = re.findall(r'\d[\w]*' str)
print(result)
```

o/p: ['1st', '21st']

RE to retreive all words having 5 character length.

```
import re
str = 'one two three four five six seven & 9 10'
result = re.findall(r'\b\w{5}\b', str)
print(result)
```

o/p: ['three', 'seven']

VISION

@python.hub

RE to retrieve all words atleast 4 characters

```
import re
str = 'one two three four five six seven 8 9 10'
result = re.findall(r'\b\w{4,}\b', str)
print(result)
```

o/p : ['three', 'four', 'five', 'seven']

RE to retrieve all words having 3 or 4 or 5 characters length.

```
import re
str = 'one two three four five six seven 8 9 10'
result = re.findall(r'\b\w{3,5}\b', str)
print(result)
```

o/p : ['one', 'two', 'three', 'four', 'five', 'six', 'seven']

RE to retrieve only single digit numbers from words of a string

```
import re
str = 'one two three four five six seven 8 9 10'
result = re.findall(r'\bd\b', str)
print(result)
```

o/p : ['8', '9']

vision

@python.hub

RE to retrieve last word of string if it is starts with t.

```
import re
str = 'five seven ten'
result = re.findall(r't[\w]*\z', str)
print(result)
```

o/p: ['ten']

RE to retrieve phone number of a person.

```
import re
str = 'Mr. Arjun Rampal:9999999999'
result = re.search(r'\d+', str)
print(result.group())
```

o/p: 9999999999

RE to retrieve only name of a person from string

```
import re
str = 'Mr. Arjun Rampal:9999999999'
result = re.search(r'\D+', str)
print(result.group())
```

o/p: Mr. Arjun Rampal:

@python.hub

RE to retrieve all words starting with 'an' or 'ak'.

```
str = 'anil akhil ankita akira arati arundhati ankur anita'
result = re.findall (r'a[nk][\w]*', str)
print(result).
```

o/p: ['anil', 'akhil', 'ankita', 'akira', 'ankur', 'anita']

RE to retrieve date of births from a string

```
import re.
str = 'anil:15 21-10-1990, Rohit:9 22-11-1995 sita:22
                                    15-09-2000'
result = re.findall (r'\d{2}-\d{2}-\d{4}', str)
print(result)
```

o/p: ['21-10-1990', '22-11-1995', '15-09-2000']

RE to check whether a given string starts with 'He' or not

```
str = "Hello world"
result = re.search (r'^He', str)
if (result != None):
    print("Yes")
else:
    print ('No')
```

Alternative:

```
res = re.match (r'He.*', str)
re = re.match (r'^He.*', str)
```

o/p: Yes.

vision

@python.hub

RE to check whether a given string end with 'world' or not.

```
import re
result = re.search(r'world\Z', str)
print(result.group())
result1 = re.search(r'world$', str)
print(result1.group())
if (result != None)
        print('string ending with world')
else:
        print('string not ending with world')
```

→ Same Question but ignore case: then

```
str = "Hello  world"
result = re.search(r'world$', str, re.IGNORECASE)
```

@python.hub

Regular Expression with File :

We can use regular expression not only on individual strings but also on a file where huge data is available.

1. First we need to open a file.
   f = open (filename, 'r')
2. Read file line by line using for loop as:
   for line in f:
   res = re.findall(regular_exp, line)

→ Atlast, any elements are stored in res then print it by checking len(res)>0 as:

```
if (len(res) >0):
    print(res)
```

Python program to read email Address from text file.

```
import re
with open ('email.txt','r') as f:
    for line in f:
        res = re.findall (r'[A-Za-z][A-Za-z0-9\.]*
              @[a-z]+\.[a-z]{2,3}', line)
        #res = re.findall (r'\S+@\S+', line)
        if (len(res)>0):
            print (res)
```

O/p:

```
['abc@gmail.com', 'bbc@india.org']
```

vision

@python.hub

EXCEPTION HANDLING

What are Exceptions?

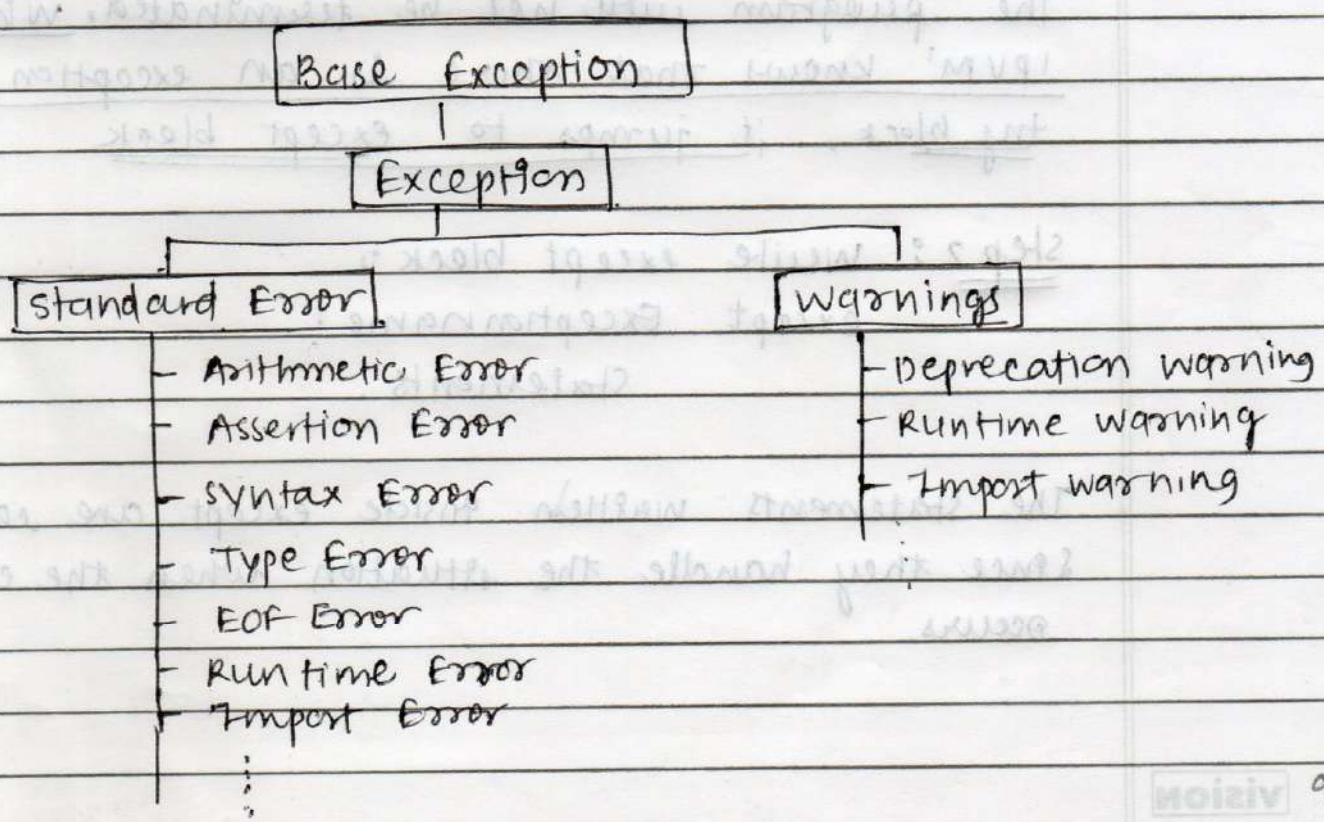
→ The runtime errors which can be handled by programmers are called exceptions.

→ That means if the programmer can guess an error in the program and he can do something to eliminate harm caused by the error, then it is called 'exception'

→ If the programmer cannot do anything in case of an error, then it is called an "error" and not an exception.

→ The exceptions already available in python are called "built-in" exceptions.

→ The base call for all built-in exceptions is 'Base Exception' class



@python.hub

→ All errors (exceptions) are defined as sub classes of Standard Error. They must be handled otherwise program will not execute.

Exception Handling:

→ The purpose of handling exceptions is to make program 'robust' - strong.

To handle exception:

Step 1: Find where is the possibility of an exception. Write these statement inside try block.

Ex: try:
    statements

Even any exception arises inside this try block, the program will not be terminated. When 'PVM' knows that there is an exception in try block, it jumps to except block

Step 2: Write except block:

except Exceptionname:
    Statements.

The statements written inside except are called "handlers". Since they handle the situation when the exception occurs.

vision

@python.hub

Step 3: Lastly, programmer should perform some cleanup tasks.
Then, write finally block (optional).

```
finally:
    statements.
```

→ Irrespective of exceptions occurs or not finally block is executed.

→ This ensures all opened files are closed and running process are terminated.

- By performing above three ~~statements~~ steps exceptions are handled.
- Here programmer is not preventing exceptions, but the programmer is avoiding any damage that happen to data and s/w.

Ex:

```
try:
    fo=open("myfile.","w")
    a,b=[int(x) for x in input("Enter:").split()]
    c=a/b
    f.write("Writing %d into my file"%c)
except ZeroDivisionError:
    print("Error")
finally:
    fo.close()
    print("file closed")
```

@python.hub

Syntax:

```
try:
    statements
except E1:
    handler 1
except E2:
    handler2
else:
    statements
finally:
    statements.
```

→ The 'try' block contains the statements where there may be one or more exceptions.

→ Two subsequent 'except' blocks handle these Exceptions.

→ When Exception (E1) occurs, 'handler 1' statements are executed.

→ When Exception (E2) occurs, 'handler 2' statements are executed. and so forth

→ If no exception raised, the statements inside 'else' block are executed.

→ Even if the exception occurs or does not occur, the code inside "finally' block is always executed.

@python.hub

Points to remember:

→ A single try block can be followed by several except blocks
→ Multiple except blocks can be used to handle multiple exceptions
→ We cannot write except blocks without a try block
→ We can write ~~except~~ try block without any except blocks
→ Else and finally blocks are not compulsory
→ When there is no exception, else block is executed after try block
→ Finally block is always executed.

Python program to handle syntax error given by eval() function.

```
try:
    date = eval(input("Enter date:"))
except SyntaxError:
    print("Invalid date entered")
else:
    print("You entered:", date)
```

O/p 1: Enter date: 2016, 10, 3
You entered: (2016, 10, 4)
Enter date: 2016, 10b, 3
Invalid date entered.

@python.hub

Python program to handle IOError produced by open() function.

```
try:
    name = input ("Enter filename:")
    f = open (name, 'r')
except IOError:
    print ('File not found')
else:
    n = len (f.readlines())
    print (name, 'has', n, 'lines')
    f.close()
```

o/p 1 : Enter filename : ex.py
ex.py has 22 lines.

o/p 2 : Enter filename : xyz
File not found

Python program to handle multiple exception :

```
def avg(list):
    tot = 0
    for x in list:
        tot += x
    avg = tot / len(list)
    return tot, avg
```

@python.hub

```
try:
    k = input("Enter space seperated values:").split()
    t,a = avg(k)
    print('Total = %d, Average = %d'.format(t,a))
except TypeError:
    print('Type Error, please provide numbers only')
except ZeroDivisionError:
    print('Zero Division error, please do not give empty list.')
```

O/p 1: Enter space seperated values: 1 2 3 4 5 a
Type Error, please provide numbers only

O/p 2: Enter space seperated values:
Zero Division error, please do not give empty list

O/p 3: Enter space seperated values: 1 1 1 1
Total = 4, Average = 1

@python.hub

Except Block

→ Except block is useful to catch an exception that is raised by try block. When there is an exception in try block, then only except block is executed.
It is written in various forms:

1. **except Exceptionclass:**
To catch the exception which is raised by try block.

2. **except Exceptionclass as obj:**
We can catch the exception as an object which contains some description about exception

3 **except (Exceptionclass1, Exceptionclass2,...):**
To catch multiple Exception, we can use a single except block and write all the exceptions as a tuple inside parameter.

Try with finally block:

```
try:
    x = int(input('Enter a number:'))
    y = 1/x
finally:
    print('we are not catching the exception.')
print("The inverse is:", y)
```

vision

@python.hub

```
o/p : Enter a number : 5
      We are not catching the exception
      The inverse is : 0.2
```

Using assert statement and catching AssertionError.

```
try :
    x = int (input ('Enter a number between 5 and 10 : '))
    assert x >= 5 and x <= 10
    print ('The number entered : ', x)
except AssertionError :
    print ('The condition is not fulfilled')
```

```
o/p : Enter a number between 5 and 10 : 12
      The condition is not fulfilled.
```

Assert Statement with a message :

```
try :
    x = int (input ('Enter a number between 5 and 10 : '))
    assert x >= 5 and x <= 10 , "Your input is not correct"
    print ('The number entered : ', x)
except AssertionError as obj :
    print (obj)
```

```
o/p : Enter a number between 5 and 10 : 12
      Your input is not correct.
```

@python.hub

USER-DEFINED EXCEPTIONS

→ Like built-in exception, the programmer can also create his own exception, which are called user-defined exceptions.

→ There are lot inbuilt exception, but may be some situation, where none of exception in python are useful for programmer.
In that case, programmer has to create his own Exceptions.

→ For this, programmer has to follow these steps:

1. Since all exceptions are classes, the programmer is supposed to create his own exception as a class. Also he should make his class as a subclass of built in 'exception' class.

```
class MyException (Exception):
    def __init__ (self, arg):
        self.msg = arg
```

Hence MyException is subclass for 'Exception' class. This class has a variable msg.

2. The programmer can write his code/function when programmer suspects the possibility of exception, he should raise his own exception using 'raise' statement.

```
raise MyException ('message')
```

VISION

@python.hub

Here raise statement is raising MyException class object that contains 'message'

3. The programmer can put code in try block and accept exception through except block as..

```
try:
    code
except MyExceptten as me:
    print(me)
```

→ Here the object 'me' contains the message given in the raise statement

Program to create own exception and raise it when needed

```
class MyException(Exception):
    def __init__(self, arg):
        self.msg = arg
    def check(dict):
        for k,v in dict.items():
            print('Name = {:15s} Balance = {:10.2f}', format(k,v))
            if(v<2000.00):
                raise MyException('Balance amount is less in account of' +k)

bank = {'Raj': 5000.00, 'Vani': 8900.50, 'Ajay': 1950.00,
        'Naresh': 3000.00}
```

@python.hub

```
try:
    check(bank)
except MyException as me:
    print(me)
```

O/p :

```
Name = Raj          Balance = 5000.00
Name = Vani         Balance = 8900.50
Name = Ajay         Balance = 1990.00
Balance amount is less in account of Ajay
```

vision